

UNIT – V

Topic 1: Abstract Data Types (ADTs)

1. Introduction

Data is the most valuable part of any computer program. To store, organize, and process data efficiently, programmers use **Data Structures**.

Before implementing a data structure, we first define its behavior and operations. This logical description is called an **Abstract Data Type (ADT)**.

An ADT tells us **what operations can be performed on the data**, but it does not explain **how those operations are implemented**.

It acts as a blueprint or contract for a data structure.

2. Definition

Simple Definition

An **Abstract Data Type (ADT)** is a logical description of a data structure that defines the data and the operations that can be performed on it.

Exam Definition

An **Abstract Data Type (ADT)** is a logical model that specifies the behavior of a data structure by defining its data and operations without revealing the implementation details.

Technical Definition

An ADT is an implementation-independent specification of a data structure that separates the interface from the internal implementation.

3. What is an ADT?

An ADT focuses on **what** operations are performed, not **how** they are implemented.

It provides a set of operations that users can perform while hiding the internal working of the data structure.

For example, a **Stack ADT** provides operations such as:

- push()
- pop()

- peek()
- isEmpty()

The user can use these operations without knowing whether the stack is implemented using an array or a linked list.

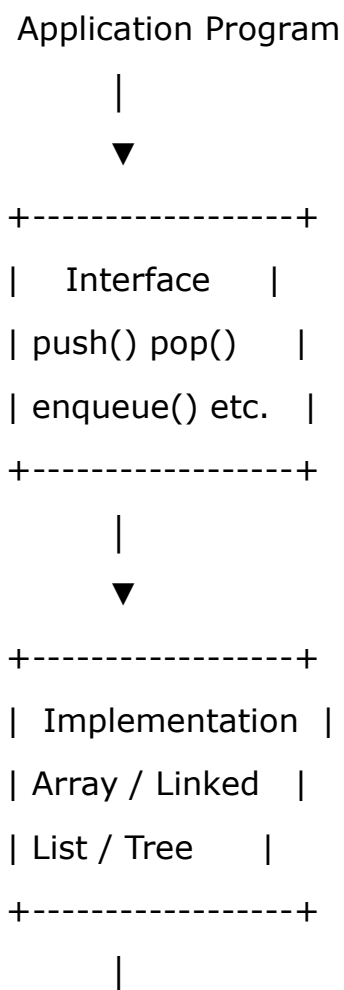
4. Basic Idea of ADT

The main idea of an ADT is to separate the **Interface** from the **Implementation**.

- **Interface** → Operations visible to the user.
- **Implementation** → Internal code and memory structure hidden from the user.

The user only interacts with the interface.

5. Architecture of ADT





Memory

6. Components of an ADT

Every ADT consists of two main components.

A. Data Representation

It describes:

- Type of data
- Logical organization of data
- Structure of data

Examples:

- Integer
 - Character
 - Float
 - String
-

B. Operations on Data

Operations define what actions can be performed on the data.

Example: Stack ADT

- push(item)
- pop()
- peek()
- isEmpty()

Example: Queue ADT

- enqueue(item)
 - dequeue()
 - front()
 - rear()
-

7. Characteristics of ADT

1. Abstraction

Only important operations are shown to the user.

Implementation details remain hidden.

2. Encapsulation

Data and operations are combined into one logical unit.

Only authorized operations can access the data.

3. Implementation Independence

The same ADT can have different implementations.

Example:

A Stack can be implemented using:

- Array
- Linked List

The user interface remains the same.

4. Modularity

A program is divided into smaller independent modules.

Each module performs a specific task.

5. Reusability

The same ADT can be used in different software applications without rewriting the code.

6. Flexibility

The internal implementation can be changed without affecting the application program.

8. Why Do We Use ADTs?

ADTs are used because they:

- Hide implementation details.
 - Reduce program complexity.
 - Improve software quality.
 - Make programs reusable.
 - Improve maintainability.
 - Support modular programming.
 - Make debugging easier.
-

9. Real-Time Examples

Example 1: ATM Machine

An ATM provides operations such as:

- Withdraw Money
- Deposit Money
- Check Balance
- Mini Statement

The customer uses these operations without knowing the internal implementation.

This is an example of an ADT.

Example 2: Mobile Phone

A smartphone provides operations like:

- Calling
- Messaging
- Internet Browsing
- Camera

The user uses these features without knowing how they are implemented internally.

Example 3: Remote Control

A TV remote has buttons like:

- Power ON/OFF
- Volume Up
- Volume Down
- Channel Change

The user only presses the buttons.

The internal electronic circuit remains hidden.

10. Working of ADT

An ADT works at three different levels.

Level 1: Abstract (User) Level

The programmer uses predefined operations such as:

- push()
- pop()
- enqueue()
- dequeue()

The programmer does not know how these operations are implemented.

Level 2: Implementation Level

The programmer implements the operations using data structures such as:

- Arrays
 - Linked Lists
-

Level 3: Machine Level

The compiler converts the program into machine language.

Memory allocation and execution are handled by the operating system and hardware.

11. Examples of ADTs

Common examples of ADTs include:

- Stack
- Queue
- Linked List
- Tree
- Graph
- Priority Queue

12. Advantages of ADTs

- Simplifies complex data management.
- Hides implementation details.
- Improves modularity.
- Improves code reusability.
- Easy to maintain.
- Easy to modify.
- Supports Object-Oriented Programming.
- Reduces programming errors.
- Makes software more reliable.
- Increases flexibility.

13. Disadvantages of ADTs

- Slight performance overhead due to abstraction.
- Designing an ADT requires careful planning.
- Beginners may find ADTs difficult to understand.
- Poor interface design can affect the entire application.

14. Applications of ADTs

Application Area	ADT Used	Purpose
Operating Systems	Queue, Stack	Process Scheduling

Application Area	ADT Used	Purpose
Compilers	Stack, Tree	Expression Evaluation
Databases	Tree, Graph	Data Indexing
Networking	Graph	Routing Algorithms
Web Browsers	Stack	Back and Forward Navigation
Text Editors	Stack	Undo and Redo
Games	Stack, Linked List	Undo Operations
GPS Navigation	Graph	Shortest Path

15. ADT vs Data Structure

Abstract Data Type	Data Structure
Logical concept	Physical implementation
Defines what operations are allowed	Defines how operations are implemented
Independent of programming language	Depends on implementation
User view	Programmer view

16. Python Syntax (Simple ADT Example)

```
class Stack:
    def push(self, item):
        pass

    def pop(self):
        pass

    def peek(self):
        pass
```

```
def isEmpty(self):  
    pass
```

This class defines the operations of a Stack ADT. The implementation details are hidden.

17. Key Points

- ADT stands for **Abstract Data Type**.
 - ADT is a logical model.
 - It defines **what** operations are performed.
 - It hides **how** operations are implemented.
 - ADT separates interface from implementation.
 - It supports abstraction and encapsulation.
 - Common ADTs include Stack, Queue, Linked List, Tree, and Graph.
-

18. Viva Questions

Q1. What is an Abstract Data Type?

Answer: An ADT is a logical description of a data structure that defines its data and operations without showing implementation details.

Q2. What is the main purpose of an ADT?

Answer: To hide implementation details and provide a simple interface.

Q3. Give two examples of ADTs.

Answer: Stack and Queue.

Q4. What are the two components of an ADT?

Answer: Data Representation and Operations.

Q5. What is abstraction?

Answer: Abstraction hides internal implementation details and shows only the required operations.

Topic 2: LINKED LIST

PART – 1

Introduction to Linked Lists

1. Definition

A **Linked List** is a **linear data structure** in which data elements are stored as **nodes**. Each node contains two parts:

1. Data
2. Address (Pointer/Reference) of the next node

Unlike an array, the nodes of a linked list are **not stored in continuous memory locations**. Instead, they are connected using pointers.

Simple Definition

A Linked List is a collection of connected nodes where each node stores **data** and the **address of the next node**.

Technical Definition

A Linked List is a dynamic linear data structure consisting of nodes connected through references, allowing efficient insertion and deletion without requiring contiguous memory.

2. Concept of Linked List

A Linked List stores information in the form of nodes.

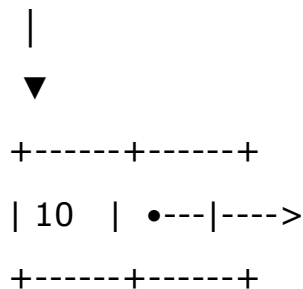
Each node knows where the next node is located.

The first node is accessed using a special pointer called **Head**.

The last node points to **NULL (None)**, indicating the end of the list.

Flow

Head





```
+-----+-----+  
| 20   | •---|---->  
+-----+-----+
```



```
+-----+-----+  
| 30   | NULL |  
+-----+-----+
```

3. Why Do We Need Linked Lists?

Arrays have some limitations:

- Fixed size
- Memory wastage
- Insertion requires shifting elements
- Deletion requires shifting elements

Linked Lists solve these problems by allowing memory to be allocated whenever a new node is created.

4. Why Use Linked Lists?

Reason	Explanation
Dynamic Size	Can grow or shrink during execution.
Easy Insertion	No shifting of elements is required.
Easy Deletion	Nodes can be removed without moving other elements.
Efficient Memory Usage	Memory is allocated only when required.

Reason	Explanation
Flexibility	Suitable for dynamic applications.

5. Characteristics of Linked Lists

- Dynamic memory allocation.
 - Nodes are stored at different memory locations.
 - No contiguous memory is required.
 - Sequential access only.
 - Supports efficient insertion.
 - Supports efficient deletion.
 - Uses pointers (references).
 - Size changes during runtime.
-

6. Components of a Linked List Node

Every node contains two important parts.

A) Data

Stores the actual information.

Examples:

- Student Name
 - Roll Number
 - Employee ID
 - Product Name
 - Marks
 - Salary
-

B) Pointer (Reference)

Stores the memory address of the next node.

It creates a connection between two nodes.

Without the pointer, the nodes cannot be linked together.

7. Structure of a Node

```
+-----+
|  DATA | NEXT  |
+-----+
```

Example

```
+-----+-----+
| 100  | 2500  |
+-----+-----+
```

Here,

100 → Data

2500 → Address of next node

8. Memory Representation

Unlike arrays, linked list nodes are stored in different memory locations.

Head

|
▼

Address : 3200

```
+-----+-----+
| 15   | 3600  |
+-----+-----+
```

|
▼

Address : 3600

```
+-----+-----+
| 25   | 4200  |
+-----+-----+
```

|



Address : 4200

```
+-----+-----+
| 40   | NULL   |
+-----+-----+
```

Observation

- Nodes are stored at different memory addresses.
- Each node stores the address of the next node.
- The last node stores NULL.

9. Head Node

The **Head** is a special pointer that stores the address of the first node.

Without the Head node, the linked list cannot be accessed.

Head

|



10 → 20 → 30 → NULL

10. Tail Node

The last node is called the **Tail Node**.

Its next pointer always contains **NULL**.

10 → 20 → 30 → NULL



Tail Node

11. How a Linked List Works

Step 1

Create a new node.



Step 2

Store the data inside the node.



Step 3

Store the address of the next node.



Step 4

Repeat the process for all nodes.



Step 5

The last node points to NULL.

12. Dynamic Memory Allocation

Memory is allocated only when a new node is created.

Example

Student joins



New Node Created



Memory Allocated

Advantages

- No memory wastage.
- Flexible size.
- Efficient memory utilization.

13. Sequential Access

Linked Lists support only **sequential access**.

To reach the fourth node, we must visit the first, second, and third nodes.

Head



Node1



Node2



Node3



Node4

Direct indexing like list[3] is **not possible** in a linked list.

14. Real-Time Examples

Example 1: Train Compartments

Engine



Coach1



Coach2



Coach3

Each coach is connected to the next coach.

Example 2: Music Playlist

Song A



Song B



Song C



Song D

Clicking **Next** moves to the next song.

Example 3: Browser History

Google



ChatGPT



GitHub



YouTube

Each webpage acts as a node.

Example 4: Task Manager

Task1



Task2



Task3

Tasks are linked one after another.

15. Applications of Linked Lists

- Memory Management
- Operating Systems
- Browser History
- Undo and Redo Operations
- Music and Video Playlists
- GPS Navigation Systems
- Task Scheduling
- Graph Implementation
- Polynomial Representation
- Stacks and Queues

16. Advantages of Linked Lists

- Dynamic memory allocation.
- Easy insertion of new nodes.
- Easy deletion of nodes.
- No shifting of elements.
- Better memory utilization.

- Suitable for dynamic applications.
 - Flexible data structure.
-

17. Disadvantages of Linked Lists

- Extra memory is required for pointers.
 - Random access is not possible.
 - Searching is slower than arrays.
 - Traversal is sequential.
 - Pointer handling makes implementation slightly complex.
-

18. Key Points to Remember

- Linked List is a linear data structure.
 - It consists of nodes.
 - Every node has Data and Pointer.
 - Nodes are stored in different memory locations.
 - Head stores the address of the first node.
 - Tail node points to NULL.
 - Memory is allocated dynamically.
 - Supports efficient insertion and deletion.
 - Sequential access only.
 - Widely used in stacks, queues, graphs, and memory management.
-

19. Viva Questions

Q1. What is a Linked List?

Answer: A Linked List is a linear data structure where each node contains data and the address of the next node.

Q2. What are the two components of a node?

Answer: Data and Pointer (Reference).

Q3. What is a Head node?

Answer: The Head node stores the address of the first node in the linked list.

Q4. What is a Tail node?

Answer: The Tail node is the last node, and its next pointer contains NULL.

Q5. Why is a Linked List called a dynamic data structure?

Answer: Because memory is allocated dynamically during program execution.

Q6. Can Linked Lists perform random access?

Answer: No. Linked Lists support only sequential access.

Q7. Which is faster for insertion: Array or Linked List?

Answer: Linked List is generally faster because elements do not need to be shifted.

Topic 2: LINKED LIST

PART – 2

Types of Linked Lists (Singly, Doubly and Circular Linked Lists)

1. Introduction to Types of Linked Lists

A Linked List can be organized in different ways depending on how the nodes are connected. Based on the connection between nodes, Linked Lists are classified into three main types.

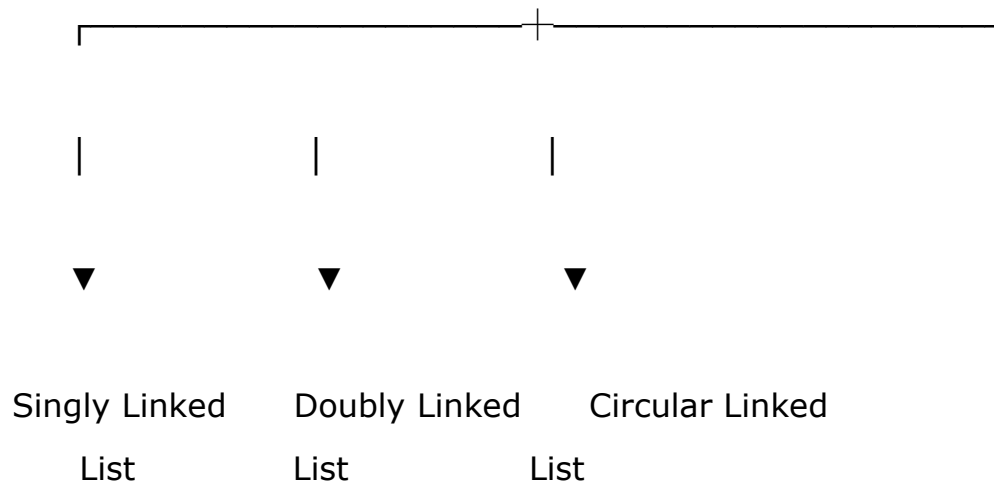
1. Singly Linked List (SLL)
2. Doubly Linked List (DLL)
3. Circular Linked List (CLL)

Each type has its own structure, advantages, disadvantages, and applications.

2. Classification of Linked Lists

LINKED LIST

|



3. Singly Linked List (SLL)

Definition

A **Singly Linked List** is a linear data structure in which every node contains two parts:

- Data
- Address of the Next Node

Each node points only to the next node.

Traversal is possible only in the forward direction.

Structure

Head

|

▼

```

+-----+-----+   +-----+-----+   +-----+-----+
| Data  | Next  | --> | Data  | Next  | --> | Data  | NULL  |
+-----+-----+   +-----+-----+   +-----+-----+

```

Example

Head

|



10 → 20 → 30 → 40 → NULL

Characteristics

- Each node has one pointer.
- Forward traversal only.
- Last node points to NULL.
- Simple structure.
- Less memory usage.

Advantages

- ✓ Easy to implement.
- ✓ Uses less memory.
- ✓ Faster insertion.
- ✓ Faster deletion.
- ✓ Suitable for simple applications.

Disadvantages

- ✗ Cannot move backward.
- ✗ Searching takes more time.
- ✗ Reverse traversal is difficult.

Real-Time Examples

- Music Playlist
- Train Compartments

- Browser Forward Navigation
- Student Attendance List

4. Doubly Linked List (DLL)

Definition

A **Doubly Linked List** is a linked list where every node contains three parts.

- Previous Pointer
- Data
- Next Pointer

It allows traversal in both forward and backward directions.

Structure

NULL

|



+-----+-----+-----+

|Prev |Data |Next |

+-----+-----+-----+

↕

+-----+-----+-----+

|Prev |Data |Next |

+-----+-----+-----+

↕

```

+-----+-----+-----+
|Prev |Data |NULL |
+-----+-----+-----+

```

Example

NULL ← 10 ⇌ 20 ⇌ 30 ⇌ 40 → NULL

Characteristics

- Two pointers in every node.
- Forward traversal.
- Backward traversal.
- Easy deletion.
- Easy insertion.

Advantages

- ✓ Supports forward traversal.
- ✓ Supports backward traversal.
- ✓ Easy node deletion.
- ✓ Suitable for browser history.
- ✓ Better navigation.

Disadvantages

- ✗ Uses extra memory.
- ✗ More pointer manipulation.
- ✗ Slightly complex implementation.

Real-Time Examples

- Browser Back and Forward buttons
- Image Gallery

- Music Player Previous and Next
- Undo and Redo Operations

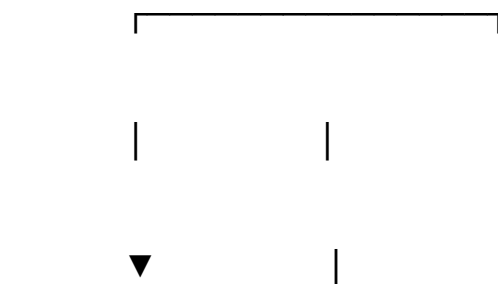
5. Circular Linked List (CLL)

Definition

A **Circular Linked List** is a linked list in which the last node points back to the first node instead of NULL.

It forms a complete circle.

Structure



10 → 20 → 30 → 40



Characteristics

- No NULL at the end.
- Forms a loop.
- Continuous traversal is possible.
- Can be singly circular or doubly circular.

Advantages

- ✓ Continuous processing.

- ✓ Efficient for cyclic operations.
- ✓ Suitable for Round Robin Scheduling.
- ✓ Better resource utilization.

Disadvantages

- ✗ Traversal is slightly difficult.
- ✗ Infinite loop may occur.
- ✗ Complex implementation.

Real-Time Examples

- Traffic Signal Control
- Round Robin CPU Scheduling
- Multiplayer Games
- Circular Queue
- Music Playlist Repeat Mode

6. Comparison Between the Three Types

Feature	Singly LL Doubly LL		Circular LL
Number of Pointers	One	Two	One or Two
Forward Traversal	Yes	Yes	Yes
Backward Traversal	No	Yes	Yes (Doubly Circular)
Last Node	NULL	NULL	Points to First Node
Memory Usage	Low	High	Medium
Complexity	Simple	Moderate	Moderate
Implementation	Easy	Medium	Medium
Performance	Good	Better Navigation Best for Cyclic Tasks	

7. Linked List vs Array

Array

Fixed Size

Continuous Memory

Random Access

Insertion is Slow

Deletion is Slow

Less Memory for Structure Extra Memory for Pointers

Linked List

Dynamic Size

Non-Continuous Memory

Sequential Access

Insertion is Fast

Deletion is Fast

8. Which Linked List Should We Choose?

Situation

Best Choice

Simple Applications

Singly Linked List

Previous and Next Navigation

Doubly Linked List

Continuous Loop Processing

Circular Linked List

Browser History

Doubly Linked List

Round Robin Scheduling

Circular Linked List

Music Playlist

Singly or Circular Linked List

9. Memory Representation

Singly Linked List

Head

↓

100 → 200 → 300 → NULL

Doubly Linked List

NULL ← 100 ⇄ 200 ⇄ 300 → NULL

Circular Linked List

100 → 200 → 300



10. Key Points

- Linked Lists are classified into three types.
- Singly Linked List has one pointer.
- Doubly Linked List has two pointers.
- Circular Linked List connects the last node to the first node.
- Singly Linked List uses less memory.
- Doubly Linked List allows backward traversal.
- Circular Linked List is suitable for repeated operations.
- Choosing the correct type depends on the application.

11. Viva Questions

Q1. How many types of Linked Lists are there?

Answer: Three types: Singly, Doubly, and Circular Linked Lists.

Q2. Which Linked List supports backward traversal?

Answer: Doubly Linked List.

Q3. Which Linked List has no NULL pointer at the end?

Answer: Circular Linked List.

Q4. Which Linked List uses the least memory?

Answer: Singly Linked List.

Q5. Which Linked List is used in browser history?

Answer: Doubly Linked List.

Q6. Which Linked List is used in Round Robin Scheduling?

Answer: Circular Linked List.

Absolutely. Below is **PART-3** in a **Microsoft Word-friendly format**. You can **copy and paste directly into MS Word** without formatting issues. The code blocks, headings, and spacing are designed to remain clean.

Topic 2: LINKED LIST

PART – 3

Singly Linked List Implementation in Python

1. Introduction

After learning the basic concepts and types of Linked Lists, the next step is to understand how to implement a **Singly Linked List** using Python.

In Python, a linked list is created using **classes and objects**. Each node is represented as an object containing data and a reference to the next node.

2. Node Structure

A **Node** is the basic building block of a Linked List.

Each node contains:

- Data
- Next Pointer (Reference)

Diagram

```
+-----+
|  DATA | NEXT  |
+-----+
```

Example

```
+-----+
| 100  | None  |
+-----+
```

3. Python Program to Create a Node

class Node:

```
def __init__(self, data):  
    self.data = data  
    self.next = None
```

Program Explanation

Line 1

class Node:

Creates a class named **Node**.

Line 2

```
def __init__(self, data):
```

This is the constructor.

It is automatically called whenever a new node is created.

Line 3

```
self.data = data
```

Stores the value inside the node.

Line 4

```
self.next = None
```

Initially, the node is not connected to any other node.

Therefore, next is assigned **None**.

Memory Representation

Node

```
+-----+
| Data = 100      |
| Next = None     |
+-----+
```

Example

```
node1 = Node(100)
```

```
print(node1.data)
```

```
print(node1.next)
```

Output

```
100
```

```
None
```

4. Creating a Linked List Class

A Linked List requires a Head pointer.

The Head stores the address of the first node.

Python Program

```
class LinkedList:
```

```
    def __init__(self):
        self.head = None
```

Explanation

```
class LinkedList:
```

Creates a Linked List class.

```
def __init__(self):
```

Constructor of Linked List.

```
self.head = None
```

Initially the list is empty.

Therefore Head = None.

Diagram

Head

↓

None

This means the Linked List is empty.

5. Creating the First Node

Now let us create our first node.

```
ll = LinkedList()
```

```
first = Node(10)
```

```
ll.head = first
```

Explanation

Step 1

```
ll = LinkedList()
```

Creates an empty Linked List.

Step 2

```
first = Node(10)
```

Creates a node containing value 10.

Step 3

```
ll.head = first
```

Head now points to the first node.

Diagram

Head

↓

```
+-----+
| 10 | None      |
+-----+
```

6. Adding Second Node Manually

```
second = Node(20)
```

```
first.next = second
```

Diagram

Head

↓

```
10 -----> 20 -----> None
```

Adding Third Node

```
third = Node(30)
```

```
second.next = third
```

Diagram

Head

↓

10 -----> 20 -----> 30 -----> None

Complete Program

```
class Node:
```

```
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class LinkedList:
```

```
    def __init__(self):
        self.head = None
```

```
ll = LinkedList()
```

```
first = Node(10)
```

```
second = Node(20)
```

```
third = Node(30)
```

```
ll.head = first
```

```
first.next = second
```

```
second.next = third
```

Memory Representation

Head

↓

```
+-----+-----+  
| 10    | •---|---->  
+-----+-----+
```

↓

```
+-----+-----+  
| 20    | •---|---->  
+-----+-----+
```

↓

```
+-----+-----+  
| 30    | None  |  
+-----+-----+
```

7. Traversing a Linked List

Traversal means visiting each node one by one from the beginning to the end.

Algorithm

Step 1

Start from Head.

↓

Step 2

Print current node data.

↓

Step 3

Move to next node.

↓

Step 4

Repeat until None.

Python Program

```
def traverse(self):
```

```
    temp = self.head
```

```
    while temp:
```

```
        print(temp.data)
```

```
        temp = temp.next
```

Explanation

```
temp = self.head
```

Starts from the first node.

while temp:

Continues until temp becomes None.

print(temp.data)

Displays the current node data.

temp = temp.next

Moves to the next node.

Example

Head

↓

10 → 20 → 30 → None

Output

10

20

30

Time Complexity

Operation Complexity

Traversal $O(n)$

Space Complexity

$O(1)$

No extra memory is used.

Advantages of Traversal

- Visits every node.
 - Displays all data.
 - Used before searching.
 - Used before deletion.
 - Used before updating.
-

Key Points

- ✓ Node is the basic building block.
 - ✓ Every node stores Data and Next.
 - ✓ Head stores the first node.
 - ✓ Linked List starts from Head.
 - ✓ Traversal visits every node.
 - ✓ Traversal stops when Next becomes None.
-

Viva Questions

Q1. What is a Node?

Answer: A Node is the basic unit of a Linked List that stores data and the address of the next node.

Q2. What is the purpose of Head?

Answer: Head stores the address of the first node.

Q3. Why is Next initialized as None?

Answer: Because the node is not connected to any other node initially.

Q4. What is Traversal?

Answer: Traversal is the process of visiting every node in a Linked List.

Q5. What is the time complexity of Traversal?

Answer: $O(n)$

Topic 2: LINKED LIST

PART – 4

Operations on Singly Linked List

1. Introduction

A Linked List becomes useful only when we perform different operations on it.

The most common operations are:

- Insertion
- Deletion
- Traversal

In this part, we will learn **Insertion Operations** step by step.

Types of Insertion

There are three types of insertion in a Singly Linked List.

1. Insertion at Beginning
 2. Insertion at End
 3. Insertion After a Specific Node
-

2. Insertion at Beginning

Definition

Insertion at Beginning means adding a new node as the **first node** of the Linked List.

The new node becomes the new Head.

Before Insertion

Head

↓

10 → 20 → 30 → None

After Insertion

Head

↓

5 → 10 → 20 → 30 → None

Algorithm

Step 1

Create a new node.

↓

Step 2

Store data.

↓

Step 3

Point new node to current Head.

↓

Step 4

Update Head to the new node.

Python Program

```
def insert_begin(self, data):  
  
    new_node = Node(data)  
  
    new_node.next = self.head  
  
    self.head = new_node
```

Program Explanation

Line 1

```
new_node = Node(data)
```

Creates a new node.

Line 2

```
new_node.next = self.head
```

The new node points to the current Head.

Line 3

```
self.head = new_node
```

The new node becomes the Head.

Output Example

Before

10 → 20 → 30 → None

After

5 → 10 → 20 → 30 → None

Time Complexity

$O(1)$

3. Insertion at End

Definition

Insertion at End means adding a new node as the last node of the Linked List.

Before

10 → 20 → 30 → None

After

10 → 20 → 30 → 40 → None

Algorithm

Step 1

Create a new node.

↓

Step 2

Move to the last node.

↓

Step 3

Change last node's next to the new node.

↓

Step 4

New node points to None.

Python Program

```
def insert_end(self, data):
```

```
    new_node = Node(data)
```

```
if self.head is None:

    self.head = new_node

    return

temp = self.head

while temp.next:

    temp = temp.next

temp.next = new_node
```

Explanation

Creates a new node.
If the list is empty,
Head points to the new node.
Otherwise,
Move to the last node.
Connect the last node to the new node.

Output

Before
10 → 20 → 30 → None
After
10 → 20 → 30 → 40 → None

Time Complexity

$O(n)$

4. Insertion After a Specific Node

Definition

A new node is inserted after a particular node in the Linked List.

Before

10 → 20 → 30 → None

Insert 25 after 20.

After

10 → 20 → 25 → 30 → None

Algorithm

Step 1

Find the required node.

↓

Step 2

Create a new node.

↓

Step 3

Store next address.

↓

Step 4

Connect new node.

Python Program

```
def insert_after(self, prev_data, data):
```

```
temp = self.head

while temp and temp.data != prev_data:

    temp = temp.next

if temp is None:

    print("Node not found")

    return

new_node = Node(data)

new_node.next = temp.next

temp.next = new_node
```

Explanation

Search the given node.

If found,

Create a new node.

Connect the new node with the next node.

Update the previous node.

Output

Before

10 → 20 → 30 → None

After

10 → 20 → 25 → 30 → None

Comparison of Insertion Operations

Operation Time Complexity

Beginning $O(1)$

End $O(n)$

After Node $O(n)$

Real-Time Examples

Insertion at Beginning

Emergency patient is added to the front of the waiting list.

Insertion at End

A new student joins at the end of the attendance list.

Insertion After a Node

A new train coach is attached after a specific coach.

Key Points

- Beginning insertion is the fastest.
 - End insertion requires traversal.
 - After-node insertion requires searching.
 - Pointer updates are very important.
 - Incorrect pointer assignment may break the Linked List.
-

Viva Questions

Q1. How many insertion operations are possible in a Singly Linked List?

Answer: Three.

Q2. Which insertion operation is fastest?

Answer: Insertion at Beginning.

Q3. Why is insertion at End slower?

Answer: Because we must traverse to the last node.

Q4. What is the time complexity of insertion at Beginning?

Answer: $O(1)$

Q5. What is the time complexity of insertion at End?

Answer: $O(n)$

Topic 2: LINKED LIST

PART –5

Deletion Operations, Searching, Updating, Complete Program, Applications & Revision

1. Deletion in Linked List

Definition

Deletion means removing an existing node from the Linked List.

Deletion can be performed in three ways:

1. Delete at Beginning
 2. Delete at End
 3. Delete a Specific Node
-

2. Deletion at Beginning

Definition

Deletion at Beginning means removing the **first node** of the Linked List.

The second node becomes the new Head.

Before Deletion

Head

↓

10 → 20 → 30 → None

After Deletion

Head

↓

20 → 30 → None

Algorithm

Step 1

Check whether the list is empty.

↓

Step 2

Move Head to the second node.

↓

Step 3

Delete the first node.

Python Program

```
def delete_begin(self):
```

```
if self.head is None:  
    print("List is Empty")  
    return  
  
self.head = self.head.next
```

Output

Before

10 → 20 → 30 → None

After

20 → 30 → None

Time Complexity

O(1)

3. Deletion at End

Definition

Delete the last node of the Linked List.

Before

10 → 20 → 30 → None

After

10 → 20 → None

Algorithm

Step 1

Move to the second last node.

↓

Step 2

Set its next pointer to None.

Python Program

```
def delete_end(self):  
  
    if self.head is None:  
        return  
  
    if self.head.next is None:  
        self.head = None  
        return  
  
    temp = self.head  
  
    while temp.next.next:  
        temp = temp.next  
  
    temp.next = None
```

Time Complexity

$O(n)$

4. Delete a Specific Node

Definition

Delete the node containing a particular value.

Before

10 → 20 → 30 → 40

Delete **30**

After

10 → 20 → 40

Python Program

```
def delete_node(self, value):

    temp = self.head

    if temp and temp.data == value:
        self.head = temp.next
        return

    prev = None

    while temp and temp.data != value:
        prev = temp
        temp = temp.next

    if temp is None:
        print("Node Not Found")
        return

    prev.next = temp.next
```

Time Complexity

O(n)

5. Searching

Definition

Searching means finding whether an element exists in the Linked List.

Algorithm

Start from Head

↓

Compare each node

↓

If found

Display "Element Found"

↓

Otherwise

Display "Not Found"

Python Program

```
def search(self, value):
```

```
    temp = self.head
```

```
    while temp:
```

```
        if temp.data == value:
```

```
            print("Element Found")
```

```
            return
```

```
        temp = temp.next
```

```
    print("Element Not Found")
```

Example

Search = 20

Output

Element Found

Time Complexity

$O(n)$

6. Updating

Definition

Updating means changing the value stored inside a node.

Before

10 → 20 → 30

Update **20** to **25**

After

10 → 25 → 30

Python Program

```
def update(self, old, new):
```

```
    temp = self.head
```

```
    while temp:
```

```
        if temp.data == old:
```

```
            temp.data = new
```

```
return
```

```
temp = temp.next
```

7. Complete Linked List Program

```
class Node:
```

```
    def __init__(self,data):
```

```
        self.data=data
```

```
        self.next=None
```

```
class LinkedList:
```

```
    def __init__(self):
```

```
        self.head=None
```

```
    def insert_end(self,data):
```

```
        new=Node(data)
```

```
        if self.head is None:
```

```
            self.head=new
```

```
        return
```

```
temp=self.head
```

```
while temp.next:
```

```
    temp=temp.next
```

```
temp.next=new
```

```
def display(self):
```

```
    temp=self.head
```

```
    while temp:
```

```
        print(temp.data,end=" -> ")
```

```
        temp=temp.next
```

```
    print("None")
```

```
ll=LinkedList()
```

```
ll.insert_end(10)
```

```
ll.insert_end(20)
```

```
ll.insert_end(30)
```

ll.display()

Output

10 -> 20 -> 30 -> None

8. Real-Time Applications

Linked Lists are used in:

- Browser History
 - Music Playlist
 - Image Viewer
 - Undo and Redo Operations
 - Memory Management
 - Operating Systems
 - GPS Navigation
 - Polynomial Representation
 - Graph Implementation
 - Stack and Queue Implementation
-

9. Advantages

- Dynamic memory allocation.
 - Easy insertion.
 - Easy deletion.
 - Better memory utilization.
 - Flexible size.
 - No shifting of elements.
-

10. Disadvantages

- Extra memory for pointers.

- Sequential access only.
- Searching is slower.
- No direct indexing.

11. Time Complexity Summary

Operation	Time Complexity
Traversal	$O(n)$
Searching	$O(n)$
Insert at Beginning	$O(1)$
Insert at End	$O(n)$
Delete at Beginning	$O(1)$
Delete at End	$O(n)$
Delete Specific Node	$O(n)$
Update	$O(n)$

12. Key Points

- Linked List is a dynamic linear data structure.
 - Each node contains Data and Next Pointer.
 - Nodes are stored in different memory locations.
 - Head stores the address of the first node.
 - Tail points to None.
 - Three types: Singly, Doubly, Circular.
 - Main operations: Traversal, Insertion, Deletion, Searching, Updating.
 - Better than arrays for frequent insertions and deletions.
-

13. Viva Questions

Q1. What is deletion in a Linked List?

Answer: It is the process of removing a node from the Linked List.

Q2. Which deletion operation is the fastest?

Answer: Deletion at the Beginning ($O(1)$).

Q3. Why is searching $O(n)$?

Answer: Because nodes are accessed sequentially.

Q4. What is updating?

Answer: Updating means changing the value stored in an existing node.

Q5. Which data structure is better for frequent insertion and deletion?

Answer: Linked List.

1. Static Memory Allocation vs Dynamic Memory Allocation

Static Memory Allocation

Static Memory Allocation means memory is allocated before the program starts executing.

The size is fixed and cannot be changed during program execution.

Examples:

- Arrays
- Static Variables

Advantages

- Fast access to memory.
- Easy to implement.
- Less overhead.

Disadvantages

- Fixed size.
- Memory wastage may occur.
- Cannot increase memory during execution.

Dynamic Memory Allocation

Dynamic Memory Allocation means memory is allocated while the program is running.

The size can increase or decrease according to the requirement.

Examples:

- Linked List
- Trees
- Graphs

Advantages

- Flexible size.
- Better memory utilization.
- Easy insertion and deletion.

Disadvantages

- Pointer handling is required.
- Slightly complex implementation.

Comparison Table

Static Memory	Dynamic Memory
Fixed size	Flexible size
Allocated before execution	Allocated during execution
Memory may be wasted	Better memory utilization
Example: Array	Example: Linked List

2. Self-Referential Structure

Definition

A **Self-Referential Structure** is a structure (or class) that contains a reference to another object of the same type.

In Python, the **Node** class is a self-referential structure because each node stores a reference to another Node.

Python Example

```
class Node:

    def __init__(self, data):
```

```
self.data = data
```

```
self.next = None
```

Here,

- **self.data** stores the value.
- **self.next** stores the reference to another Node.

Diagram

```
+-----+
| Data | Next Address |
+-----+
```

3. Logical Representation vs Physical Representation

Logical Representation

Logical Representation shows how nodes are connected.

Head

↓

10 → 20 → 30 → None

The user sees only the logical connection.

Physical Representation

Physical Representation shows the actual memory locations.

Address 1000

```
+-----+-----+
| 10   |2000  |
```

+-----+-----+

Address 2000

+-----+-----+

| 20 | 3000 |

+-----+-----+

Address 3000

+-----+-----+

| 30 | None |

+-----+-----+

Difference

Logical Representation Physical Representation

Shows node connections Shows memory addresses

Easy to understand Shows actual storage

User view Computer view

4. Empty Linked List

Definition

An Empty Linked List is a list that contains no nodes.

The Head pointer points to **None**.

Diagram

Head

↓

None

Python Example

```
class LinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

Initially, the linked list is empty.

5. NULL (None) Pointer

Definition

The last node in a Linked List always points to **NULL (None)**.

NULL indicates that there are no more nodes.

Diagram

10 → 20 → 30 → None

Importance

- Indicates the end of the Linked List.
 - Prevents traversal beyond the last node.
 - Stops the loop during traversal.
-

6. Pointer (Reference)

Definition

A Pointer (Reference) stores the address of the next node.

It creates a connection between two nodes.

Diagram

Node 1

Data = 10

Next = Address of Node 2

Real-Time Example

Think of railway coaches.

Each coach is connected to the next coach.

The connection acts like a pointer.

7. Linked List Creation Process

Steps

Start

↓

Create Node

↓

Allocate Memory

↓

Store Data

↓

Link Node



Repeat



Last Node → None



Stop

Explanation

- Create a node.
- Allocate memory.
- Store the data.
- Connect it to the next node.
- Repeat until all nodes are connected.
- The last node points to **None**.

8. Complete Traversal Algorithm

Algorithm

Step 1: Start.

Step 2: Check whether the list is empty.

Step 3: If empty, display "**List is Empty**" and stop.

Step 4: Otherwise, assign temp = head.

Step 5: Display temp.data.

Step 6: Move to the next node using temp = temp.next.

Step 7: Repeat Steps 5 and 6 until temp becomes None.

Step 8: Stop.

9. Pointer Change During Insertion

Before Insertion

10 → 20 → 30

Insert **25**

Pointer Changes

new.next = 30

20.next = new

After Insertion

10 → 20 → 25 → 30

10. Pointer Change During Deletion

Before

10 → 20 → 30 → 40

Delete **30**

Pointer Change

20.next = 40

After

10 → 20 → 40

11. Complete Operations Summary

Operation	Purpose	Time Complexity
Traversal	Visit all nodes	$O(n)$
Insertion at Beginning	Add first node	$O(1)$
Insertion at End	Add last node	$O(n)$
Insert After Node	Insert after a specific node	$O(n)$
Delete Beginning	Remove first node	$O(1)$
Delete End	Remove last node	$O(n)$
Delete Specific Node	Remove required node	$O(n)$
Searching	Find a node	$O(n)$
Updating	Modify node data	$O(n)$

TOPIC 3: STACKS (PART 1)

Stacks: LIFO Principle (Implementation Using List)

STACK

1. Introduction

A **Stack** is one of the most important linear data structures used in Computer Science. It stores data in a specific order where insertion and deletion are performed only at one end, called the **Top** of the stack.

A stack follows the **Last In, First Out (LIFO)** principle. This means the last element inserted into the stack is the first element removed.

Stacks are widely used in operating systems, programming languages, web browsers, compilers, expression evaluation, recursion, and many real-time applications.

2. Definition

Simple Definition

A **Stack** is a linear data structure in which insertion (**Push**) and deletion (**Pop**) are performed only at one end called the **Top**.

Technical Definition

A **Stack** is an Abstract Data Type (ADT) that follows the **Last In, First Out (LIFO)** principle, where the most recently inserted element is the first one to be removed.

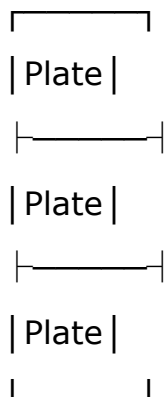
Exam Definition (2 Marks)

A **Stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle. Elements are inserted and deleted only from the top of the stack using Push and Pop operations.

3. Real-Life Examples of Stack

Example 1: Stack of Plates

Top

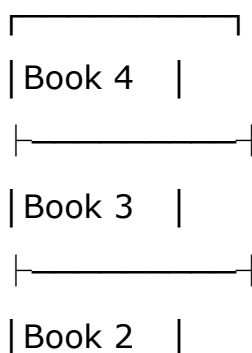


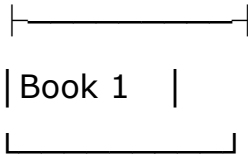
Bottom

The last plate placed on the top is the first one removed.

Example 2: Books

Top





Book 4 is removed first.

Example 3: Browser Back Button

When you visit multiple websites:

Google



YouTube



Wikipedia



ChatGPT

Pressing **Back** takes you from:

ChatGPT



Wikipedia



YouTube



Google

This follows the Stack concept.

4. Why Do We Need Stack?

Stacks help in managing data in situations where the latest information must be processed first.

They are used because they:

- Store data in an organized manner.
- Allow fast insertion and deletion.
- Simplify recursive function calls.
- Help solve many real-world problems.
- Improve program efficiency.

5. Characteristics of Stack

- It is a Linear Data Structure.
- It follows the LIFO principle.
- Insertion happens only at the Top.
- Deletion happens only at the Top.
- Only the Top element is directly accessible.
- Push and Pop operations take **O(1)** time.
- Simple and memory efficient.

6. LIFO Principle (Last In First Out)

LIFO means:

Last In → First Out

The element inserted last is removed first.

Example

Insert the numbers:

10

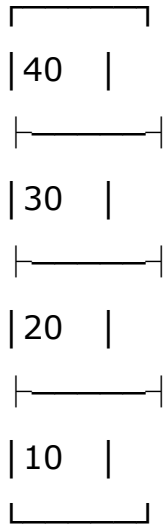
20

30

40

After pushing all elements:

Top

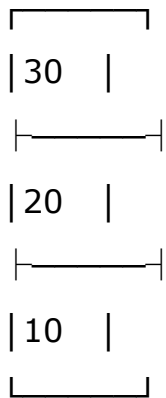


Now perform Pop.

Removed = 40

Remaining Stack

Top

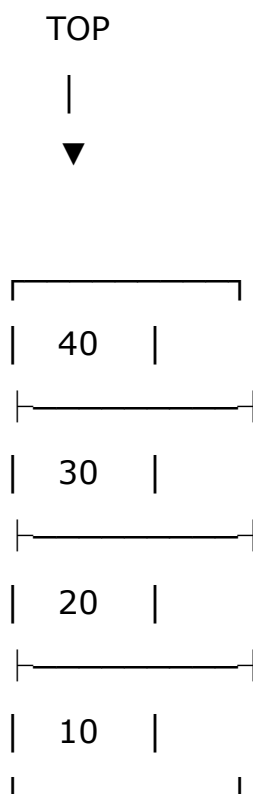


7. Stack Terminology

Term	Meaning
Stack	Collection of elements
Top	Last inserted element

Term	Meaning
Push	Insert an element
Pop	Remove the top element
Peek/Top	View the top element
Empty Stack	No elements available
Overflow	Stack is full
Underflow	Stack is empty

8. Basic Stack Structure



Only the **Top** element can be inserted or removed.

9. Push and Pop Concept

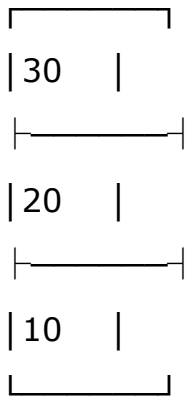
Push (Insertion)

Adds a new element to the top.

Before Push

Top

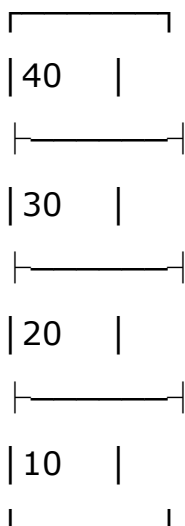
|



Push(40)

Top

|

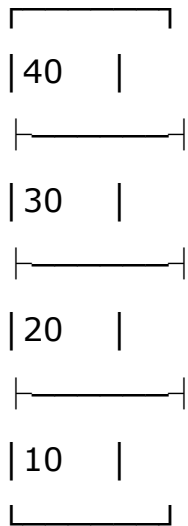


Pop (Deletion)

Removes the top element.

Before Pop

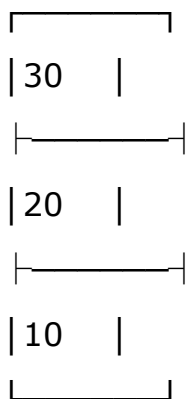
Top



After Pop

Removed = 40

Top

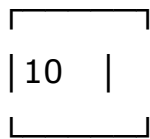


10. How Stack Works

Step 1

Push(10)

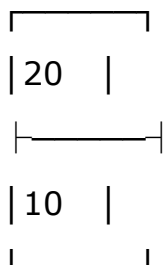
Top



Step 2

Push(20)

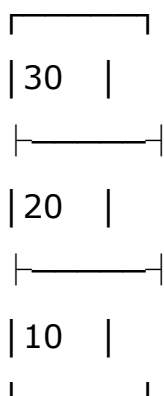
Top



Step 3

Push(30)

Top

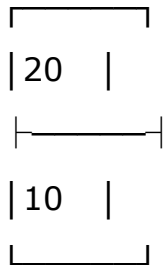


Step 4

Pop()

30 Removed

Top



11. Time Complexity of Stack Operations

Operation Time Complexity

Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
isEmpty	$O(1)$
Size	$O(1)$

12. Key Points to Remember

- Stack follows the **LIFO** principle.
- Insert operation is called **Push**.
- Delete operation is called **Pop**.
- Only the **Top** element can be accessed.
- Push and Pop operations are very fast (**$O(1)$**).
- Stacks are widely used in recursion, expression evaluation, browser history, undo/redo operations, and compilers.

TOPIC 3: STACKS (PART 2)

Stack Operations and Python Implementation Using List

13. Basic Operations on Stack

A stack supports five basic operations. These operations help us insert, remove, view, and manage elements efficiently.

The basic operations are:

- Push
- Pop
- Peek (Top)
- isEmpty()
- Size()

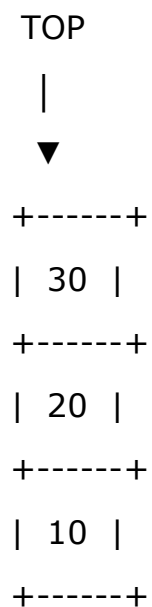
14. Push Operation

Definition

Push is the operation used to insert a new element at the **top** of the stack.

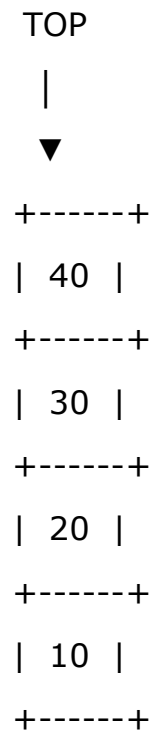
Diagram

Before Push



Push(40)

After Push



Algorithm

Step 1: Check whether the stack is full (for fixed-size stack).

Step 2: If full, display **Stack Overflow**.

Step 3: Otherwise, insert the new element at the top.

Step 4: Increase the top position.

Step 5: Stop.

Python Statement

```
stack.append(40)
```

15. Pop Operation

Definition

Pop removes the top element from the stack.

Diagram

Before Pop

TOP

|



+-----+

| 40 |

+-----+

| 30 |

+-----+

| 20 |

+-----+

| 10 |

+-----+

After Pop

Removed = 40

TOP

|



+-----+

| 30 |

+-----+

| 20 |

+-----+

| 10 |

+-----+

Algorithm

Step 1: Check whether the stack is empty.

Step 2: If empty, display **Stack Underflow**.

Step 3: Remove the top element.

Step 4: Decrease the top position.

Step 5: Return the removed element.

Python Statement

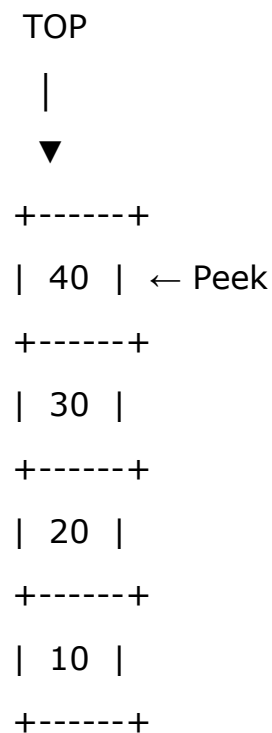
```
stack.pop()
```

16. Peek (Top) Operation

Definition

Peek returns the top element without removing it.

Diagram



Python Statement

```
stack[-1]
```

Example

```
Stack = [10, 20, 30, 40]
```

Peek = 40

The stack remains unchanged.

17. isEmpty() Operation

Definition

Checks whether the stack contains any elements.

Syntax

```
len(stack) == 0
```

Example

```
Stack = []
```

Output

True

```
Stack = [10]
```

Output

False

18. Size Operation

Definition

Returns the number of elements present in the stack.

Syntax

```
len(stack)
```

Example

```
Stack = [10,20,30]
```

Output = 3

19. Time Complexity

Operation Time Complexity

Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
isEmpty	$O(1)$
Size	$O(1)$

20. Stack Overflow

Definition

Stack Overflow occurs when we try to insert an element into a full stack.

Diagram

TOP

```
+-----+
| 50  |
+-----+
| 40  |
+-----+
| 30  |
+-----+
| 20  |
+-----+
| 10  |
+-----+
```

Push(60)

✗ Stack Overflow

Reasons

- Fixed-size stack is full.
- No extra memory is available.

21. Stack Underflow

Definition

Stack Underflow occurs when we try to remove an element from an empty stack.

Diagram

TOP

Empty Stack

Pop()

✗ Stack Underflow

22. Python Implementation Using List

Python provides a built-in **list** that can easily implement a stack.

Program

```
# Stack using List
```

```
stack = []
```

```
# Push operation
```

```
stack.append(10)
```

```
stack.append(20)
```

```
stack.append(30)
```

```
print("Stack after Push:", stack)
```

```
# Peek operation
print("Top Element:", stack[-1])

# Pop operation
removed = stack.pop()
print("Removed Element:", removed)

print("Stack after Pop:", stack)

# Size
print("Size:", len(stack))

# isEmpty
print("Is Empty:", len(stack) == 0)
```

Output

Stack after Push: [10, 20, 30]

Top Element: 30

Removed Element: 30

Stack after Pop: [10, 20]

Size: 2

Is Empty: False

23. Step-by-Step Execution

Step 1

```
stack = []
```

Output

```
[]
```

Step 2

```
stack.append(10)
```

Stack

```
[10]
```

Step 3

```
stack.append(20)
```

Stack

```
[10,20]
```

Step 4

```
stack.append(30)
```

Stack

```
[10,20,30]
```

Step 5

```
stack[-1]
```

Output

30

Step 6

```
stack.pop()
```

Output

30

Remaining Stack

[10,20]

Step 7

```
len(stack)
```

Output

2

Step 8

```
len(stack)==0
```

Output

False

24. Key Points

- Push inserts an element at the top.
 - Pop removes the top element.
 - Peek views the top element.
 - isEmpty checks whether the stack is empty.
 - Size returns the total number of elements.
 - Python lists provide an easy implementation of a stack.
 - Push and Pop operations take constant time **O(1)**.
-

TOPIC 3: STACKS (PART 3)

Complete Stack ADT Implementation Using Python List

25. Stack Implementation Using Python List

Python does not have a built-in Stack class. However, a **List** can be used as a Stack because it supports **append()** and **pop()** operations.

Program: Stack ADT Using List

class Stack:

 # Constructor

 def __init__(self):

 self.items = []

 # Push Operation

 def push(self, item):

 self.items.append(item)

 print(item, "Inserted")

```
# Pop Operation
def pop(self):
    if self.is_empty():
        print("Stack Underflow")
        return None
    return self.items.pop()
```

```
# Peek Operation
def peek(self):
    if self.is_empty():
        print("Stack is Empty")
        return None
    return self.items[-1]
```

```
# isEmpty Operation
def is_empty(self):
    return len(self.items) == 0
```

```
# Size Operation
def size(self):
    return len(self.items)
```

```
# Display Operation
def display(self):
    print("Stack:", self.items)
```

```
# Main Program
```

```
stack = Stack()

stack.push(10)
stack.push(20)
stack.push(30)

stack.display()

print("Top Element =", stack.peak())

print("Deleted =", stack.pop())

stack.display()

print("Size =", stack.size())

print("Is Empty =", stack.is_empty())
```

Output

```
10 Inserted
20 Inserted
30 Inserted
```

```
Stack: [10, 20, 30]
```

```
Top Element = 30
```

```
Deleted = 30
```

Stack: [10, 20]

Size = 2

Is Empty = False

Program Explanation (Step by Step)

Step 1

class Stack:

Creates a new class named **Stack**.

Step 2

```
def __init__(self):
```

```
    self.items = []
```

Creates an empty list.

Initially,

```
Stack = [ ]
```

Step 3

```
def push(self,item):
```

Creates the Push function.

```
self.items.append(item)
```

Adds an element at the top.

Example

Before

```
[10,20]
```

After

```
[10,20,30]
```

Step 4

`def pop(self):`

Creates the Pop function.

`return self.items.pop()`

Removes the last inserted element.

Example

Before

[10,20,30]

After

[10,20]

Removed Element

30

Step 5

`def peek(self):`

Returns the top element without removing it.

Example

Stack

[10,20,30]

Peek

30

Stack remains the same.

Step 6

```
def is_empty(self):
```

Checks whether the stack contains elements.

Returns

True

or

False

Step 7

```
def size(self):
```

Returns the number of elements.

Example

Stack

[10,20,30]

Size = 3

Step 8

```
def display(self):
```

Displays all stack elements.

Output

Stack: [10,20,30]

Dry Run

Initially

Stack

[]

Push(10)

Top

+-----+

| 10 |

+-----+

Stack

[10]

Push(20)

Top

+-----+

| 20 |

+-----+

| 10 |

+-----+

Stack

[10,20]

Push(30)

Top

+-----+

| 30 |

+-----+

| 20 |

+-----+

| 10 |

+-----+

Stack

[10,20,30]

Peek()

Returns

30

Stack remains

[10,20,30]

Pop()

Removed

30

Remaining Stack

Top

+-----+

| 20 |

+-----+

| 10 |

+-----+

Stack

[10,20]

Time Complexity

Operation Complexity

Push $O(1)$

Pop $O(1)$

Operation Complexity

Peek $O(1)$

Size $O(1)$

isEmpty $O(1)$

Advantages of List-Based Stack

- Very easy to implement.
- Uses Python built-in list.
- Push and Pop operations are fast.
- No need to write memory allocation code.
- Suitable for small and medium-sized applications.

Limitations

- Random insertion is not allowed.
- Only the top element can be accessed.
- Searching is slower because elements are accessed sequentially.
- For very large applications, linked-list implementation may be preferred.

Important Viva Questions

1. What is a Stack?

A Stack is a linear data structure that follows the LIFO principle.

2. Which Python data structure is used to implement a Stack?

Python List.

3. Which function is used for Push?

append()

4. Which function is used for Pop?

pop()

5. What is the time complexity of Push and Pop?

$O(1)$

TOPIC 3: STACKS (PART 4)

Typical Applications, Parentheses Matching, Postfix Evaluation, Comparison, Advantages & Disadvantages

26. Typical Applications of Stack

A **Stack** is used in many real-world software applications because it follows the **LIFO (Last In, First Out)** principle.

A. Program Execution (Function Calls)

Whenever a function is called, the computer stores its information in a **Call Stack**.

After the function finishes execution, it is removed from the stack.

Diagram

TOP

```
+-----+
| Function C   |
+-----+
| Function B   |
+-----+
| Function A   |
+-----+
```

Function **C** finishes first, then **B**, then **A**.

Applications

- Function calls
 - Recursive functions
 - Exception handling
-

B. Undo / Redo Operations

Applications like **MS Word**, **Photoshop**, and **Notepad++** use stacks.

Undo Operation

Action 1



Action 2



Action 3



Undo

Removes Action 3 First

Since the latest action is removed first, it follows **LIFO**.

C. Browser History

Every visited webpage is pushed into the stack.

When the user clicks **Back**, the latest webpage is removed.

Example

Google



YouTube



Wikipedia



ChatGPT

Back Button

ChatGPT

↓

Wikipedia

↓

YouTube

↓

Google

D. Parentheses Matching

Stacks are used to check whether brackets are balanced.

Example

Correct

()

[]

{}

([])

{[]}

Incorrect

(

[(

({

((

E. Expression Evaluation

Stacks are used for

- Infix to Postfix Conversion
- Infix to Prefix Conversion
- Postfix Evaluation
- Prefix Evaluation

Example

Infix

A+B

↓

Postfix

AB+

↓

Result

F. Depth First Search (DFS)

DFS uses Stack to visit graph nodes.

Example

A

↓

B

↓

D

↑

C

G. Memory Management

During program execution

- Local Variables
- Function Calls
- Return Addresses

are stored in the Stack.

H. String Reversal

Input

HELLO

Push

H

E

L

L

O

Pop

O

L

L

E

H

Output

OLLEH

27. Parentheses Matching Program

Program

```
def balanced(expression):
```

```
    stack = []
```

```
    pairs = {
```

```
        ')': '(',
```

```
        ']': '[',
```

```
        '}': '{'
```

```
    }
```

```
    for ch in expression:
```

```
    if ch in "({":
        stack.append(ch)

    elif ch in ")}":

        if not stack:
            return False

        if stack.pop() != pairs[ch]:
            return False

    return len(stack) == 0

print(balanced("{[()] }"))
print(balanced("{[ ( ] }"))
```

Output

True

False

Explanation

Expression

{ [()] }

Push

{

[

(

Closing brackets

)

]

}

Pop happens in reverse order.

All brackets match.

Hence

True

28. Postfix Expression Evaluation

Example

Expression

$23+4*$

Calculation

$2+3=5$

$5 \times 4=20$

Answer

20

Python Program

```
def evaluate():
```

```
stack=[]  
  
stack.append(2)  
  
stack.append(3)  
  
a=stack.pop()  
  
b=stack.pop()  
  
stack.append(b+a)  
  
stack.append(4)  
  
a=stack.pop()  
  
b=stack.pop()  
  
stack.append(b*a)  
  
print(stack.pop())
```

```
evaluate()
```

Output

20

29. Comparison Between Array Stack and Linked List Stack

Feature	Array/List Stack	Linked List Stack
---------	------------------	-------------------

Feature	Array/List	Stack Linked List	Stack
Memory	Contiguous		Dynamic
Push	$O(1)$		$O(1)$
Pop	$O(1)$		$O(1)$
Memory Usage	Fixed (if array)		Dynamic
Easy to Implement	Yes		Moderate
Memory Wastage	Possible		Very Less

30. Advantages of Stack

- Very easy to implement.
 - Follows LIFO principle.
 - Push and Pop operations take **$O(1)$** time.
 - Efficient memory usage.
 - Useful in recursion.
 - Used in compilers.
 - Useful in browser history.
 - Supports Undo and Redo.
 - Used in expression evaluation.
 - Helpful in DFS algorithm.
-

31. Disadvantages of Stack

- Only the top element can be accessed.
- Random access is not possible.
- Searching is difficult.
- Stack Overflow may occur.
- Stack Underflow may occur.
- Not suitable for large sequential searching.
- Cannot directly access middle elements.

32. Applications of Stack

- Function Call Management
- Recursive Programming
- Browser Navigation
- Undo / Redo Operations
- Parentheses Matching
- Expression Evaluation
- Compiler Design
- Syntax Checking
- Depth First Search (DFS)
- Memory Management
- String Reversal
- Backtracking Algorithms
- Maze Solving
- XML / HTML Tag Matching

33. Case Study 1 – Browser History Navigation

Problem

Users should be able to

- Move Back
- Move Forward

Solution

Two Stacks are used.

Back Stack

Stores previously visited pages.

Forward Stack

Stores pages that can be revisited.

Working

Suppose user visits

Google

↓

YouTube

↓

Wikipedia

↓

ChatGPT

Click **Back**

Current Page

ChatGPT

↓

Moved to

Wikipedia

ChatGPT is pushed into the **Forward Stack**.

Click **Forward**

ChatGPT is restored.

Advantages

- Fast Navigation
 - $O(1)$ Operations
 - Efficient Memory Usage
-

34. Case Study 2 – Undo / Redo in Text Editors

Applications

- MS Word
 - Photoshop
 - VS Code
 - Notepad++
-

Working

Step 1

User types

Hello

Action stored in **Undo Stack**.

Step 2

User types

World

Undo Stack becomes

Hello

↓

World

Step 3

User presses **Undo**

World is removed.

Step 4

World is stored in **Redo Stack**.

Step 5

User presses **Redo**

World is restored.

35. Important Viva Questions

1. Which principle does Stack follow?

Answer: LIFO (Last In, First Out).

2. Which operations are performed on Stack?

Answer: Push, Pop, Peek, isEmpty, and Size.

3. Which Python function performs Push?

Answer: append().

4. Which Python function performs Pop?

Answer: pop().

5. What is Stack Overflow?

Answer: Trying to insert an element into a full stack.

6. What is Stack Underflow?

Answer: Trying to remove an element from an empty stack.

7. What is the time complexity of Push and Pop?

Answer: $O(1)$.

8. Name four applications of Stack.

Answer: Browser History, Undo/Redo, Expression Evaluation, Function Calls.

Additional Notes (Textbook Topics)

1. Stack as an Abstract Data Type (ADT)

Definition

A **Stack** is an **Abstract Data Type (ADT)** that follows the **Last In First Out (LIFO)** principle.

An ADT defines **what operations** can be performed on the stack without specifying **how** they are implemented.

The Stack ADT supports the following operations:

- Push()
- Pop()
- Peek()
- isEmpty()
- Size()

A Stack can be implemented using:

- Arrays (or Python Lists)
- Linked Lists

2. Stack Representation Using Array

A stack can be implemented using an array (or Python list).

The variable **TOP** stores the index of the last inserted element.

Initially,

TOP = -1

which indicates that the stack is empty.

Diagram

Index

0

1

2

3

4

```

+-----+-----+-----+-----+
|10 |20 |30 |   |   |
+-----+-----+-----+-----+

```

TOP = 2

3. Stack Representation Using Linked List

A stack can also be implemented using a Linked List.

In this implementation:

- Every element is a node.
- The **Head** node acts as the **Top** of the stack.

Diagram

TOP

↓

```

+-----+-----+
|30   | •----|
+-----+-----+

```

↓

```

+-----+-----+
|20   | •----|
+-----+-----+

```

↓

```

+-----+-----+
|10   | None |

```

+-----+-----+

Advantages

- Dynamic memory allocation.
- No Stack Overflow due to fixed size.
- Memory is allocated only when required.

4. Push Algorithm (Pseudo Code)

Algorithm PUSH(Stack, Item)

Step 1 : Check whether the stack is full.

Step 2 : If Full

Display "Stack Overflow"

Stop

Step 3 : $TOP = TOP + 1$

Step 4 : $STACK[TOP] = Item$

Step 5 : Stop

5. Pop Algorithm (Pseudo Code)

Algorithm POP(Stack)

Step 1 : Check whether the stack is empty.

Step 2 : If Empty

Display "Stack Underflow"

Stop

Step 3 : Item = STACK[TOP]

Step 4 : TOP = TOP - 1

Step 5 : Return Item

Step 6 : Stop

6. Peek Algorithm

Algorithm PEEK(Stack)

Step 1 : Check whether the stack is empty.

Step 2 : If Empty

Display "Stack is Empty"

Step 3 : Otherwise

Display STACK[TOP]

Step 4 : Stop

7. isEmpty Algorithm

Algorithm isEmpty(Stack)

Step 1 : If TOP == -1

Return True

Step 2 : Else

Return False

Step 3 : Stop

8. Dry Run Table

Operation	Stack	TOP
Start	Empty	-1
Push(10)	10	0
Push(20)	10,20	1
Push(30)	10,20,30	2
Peek()	10,20,30	2
Pop()	10,20	1

9. Complete Execution Flow

Start

↓

Create Empty Stack

↓

Push(10)

↓

Push(20)

↓

Push(30)

↓

Peek()

↓

Pop()

↓

Display Stack

↓

Stop

10. Stack Operation Summary

Operation Purpose		Time Complexity
Push	Insert an element	$O(1)$
Pop	Remove the top element	$O(1)$
Peek	View the top element	$O(1)$
isEmpty	Check whether the stack is empty	$O(1)$
Size	Count elements	$O(1)$

11. Difference Between Overflow and Underflow

Stack Overflow

Occurs when inserting into a full stack

Happens during Push

Mainly seen in fixed-size stacks

Stack Underflow

Occurs when deleting from an empty stack

Happens during Pop

Can occur in any empty stack

12. Key Points

- Stack is an Abstract Data Type (ADT).
- Stack follows the LIFO principle.
- A Stack can be implemented using Arrays (Lists) or Linked Lists.
- The **TOP** pointer always points to the last inserted element.
- Push increases the TOP value.
- Pop decreases the TOP value.
- Overflow occurs when the stack is full.
- Underflow occurs when the stack is empty.

TOPIC 4: QUEUES (PART 1)

Queues: FIFO Principle, Introduction, Basic Concepts & Applications

UNIT – V

Topic 4: QUEUES

1. Introduction to Queue

A **Queue** is a **linear data structure** in which elements are inserted at one end and removed from the other end.

A Queue follows the **FIFO (First In, First Out)** principle.

This means the element that enters the queue first will leave the queue first.

Queues are widely used in operating systems, printers, CPU scheduling, networking, and many real-time applications.

2. Definition of Queue

Definition

A **Queue** is a linear data structure that stores elements in a sequence where:

- Insertion takes place at the **Rear**
- Deletion takes place at the **Front**

It follows the **FIFO (First In, First Out)** rule.

Simple Definition (Exam Point)

A Queue is a linear data structure that follows the FIFO principle, where the first inserted element is the first one to be removed.

3. Queue ADT (Abstract Data Type)

A Queue ADT defines the operations that can be performed on a queue without explaining how they are implemented.

Queue ADT Operations

enqueue(item)

dequeue()

front()

rear()
isEmpty()
size()

4. FIFO Principle

FIFO means:

First In → First Out

The first element inserted into the queue is removed first.

Example

Insert:

10
20
30
40

Removal Order:

10
20
30
40

Real-Life Example 1 – Ticket Counter

People Joining Queue

Ravi
Kiran
Anu

Ramesh



Ticket Counter



Service Order

Ravi

Kiran

Anu

Ramesh

The first person who joins gets the ticket first.

Real-Life Example 2 – Bus Queue

Passengers

A

B

C

D



Bus Arrives



Boarding Order

A

B

C

D

Real-Life Example 3 – Billing Counter

Customers stand in a queue.

The cashier serves customers in the order they arrived.

Real-Life Example 4 – Printer Queue

Documents are printed in the order they were submitted.

Doc1

Doc2

Doc3

↓

Printer

↓

Printed Order

Doc1

Doc2

Doc3

5. Queue Structure

A Queue has two important positions.

Front

Rear



```
+-----+-----+-----+-----+-----+
| 10  | 20  | 30  | 40  | 50  |
+-----+-----+-----+-----+-----+
```

Delete Here

Insert Here

Front

- First element
- Deletion happens here

Rear

- Last element
- Insertion happens here

6. Basic Queue Diagram

ENQUEUE



Front

Rear



```

+-----+-----+-----+-----+
| 11 | 22 | 33 | 44 |
+-----+-----+-----+-----+

```

↑

DEQUEUE

7. Queue Representation

Suppose we insert:

10

20

30

40

Queue becomes

Front

↓

10 → 20 → 30 → 40

↑

Rear

After deleting one element

Front

↓

20 → 30 → 40

↑

Rear

8. Queue Terminology

Term	Meaning
Queue	Linear data structure
FIFO	First In First Out
Front	First element
Rear	Last element
Enqueue	Insert operation
Dequeue	Delete operation
Peek/Front View	View first element
Rear()	View last element

9. Characteristics of Queue

- Linear data structure
- Follows FIFO principle
- Insertion at Rear
- Deletion at Front
- Easy to implement

- Stores data sequentially
 - Suitable for scheduling applications
-

10. Why Do We Use Queue?

Queues are used whenever tasks should be processed in the order they arrive.

Examples:

- Ticket booking
 - Printer jobs
 - CPU scheduling
 - Call centers
 - Banking systems
 - Hospital registration
 - Customer service
-

11. Advantages of Queue

- Simple to understand
 - Easy insertion and deletion
 - Maintains processing order
 - Fair scheduling
 - Useful in operating systems
 - Efficient for sequential processing
-

12. Disadvantages of Queue

- No random access
 - Searching takes more time
 - Only front element can be removed
 - Array implementation has fixed size
 - Overflow and Underflow may occur
-

13. Applications of Queue

Application	Purpose
CPU Scheduling	Executes processes in order
Printer Queue	Prints documents sequentially
Call Center	Handles customer calls
Banking System	Serves customers
Ticket Reservation Processes	booking requests
BFS Algorithm	Graph traversal
Network Packets	Data transmission
IO Buffers	Temporary data storage

14. Time Complexity Overview

Operation Time Complexity

Enqueue	$O(1)$
Dequeue	$O(1)$
Front	$O(1)$
Rear	$O(1)$
isEmpty	$O(1)$
Size	$O(1)$

15. Key Points for Exams

- Queue is a **Linear Data Structure**.
- Queue follows the **FIFO Principle**.
- Insertion is called **Enqueue**.
- Deletion is called **Dequeue**.
- Elements are inserted at the **Rear**.
- Elements are removed from the **Front**.

- Queue is used in **CPU Scheduling, Printer Queue, BFS, Networking, Ticket Reservation, and Banking Systems.**
-

TOPIC 4: QUEUES (PART 2)

Queue Operations (MS Word Friendly | Student Friendly | Exam Ready)

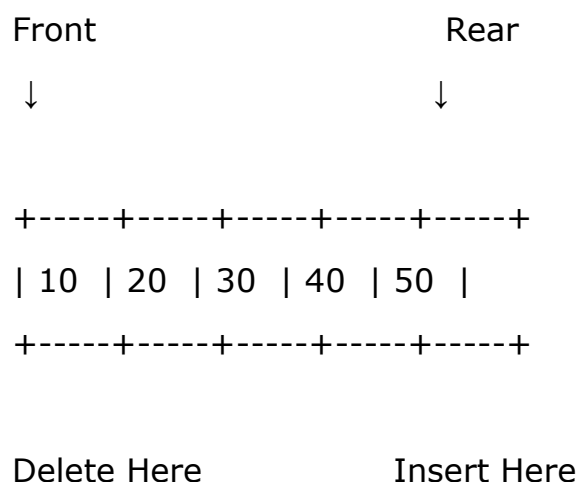
16. Operations on Queue

A Queue mainly supports the following operations:

1. Enqueue()
 2. Dequeue()
 3. Front() / Peek()
 4. Rear()
 5. isEmpty()
 6. Size()
-

Queue Operations Diagram

QUEUE



1. Enqueue Operation

Definition

Enqueue means inserting a new element at the **Rear** of the queue.

Only the rear end is used for insertion.

Algorithm

Step 1: Check whether queue is full.

Step 2: If full, display "Queue Overflow".

Step 3: Otherwise insert the element at Rear.

Step 4: Move Rear to the next position.

Step 5: Stop.

Example

Initially

Front = 10

10 → 20 → 30

Insert 40

10 → 20 → 30 → 40

Rear = 40

Step-by-Step

Step 1

Front

↓

10 → 20 → 30

Step 2

Insert 40

Step 3

Front

↓

10 → 20 → 30 → 40

↑

Rear

Real-Time Example

Imagine customers joining a bank queue.

Ravi

Kiran

Anu

↓

New Customer

Rahul

Rahul joins at the **rear**.

2. Dequeue Operation

Definition

Dequeue means removing the element from the **Front** of the queue.
Only the front element can be removed.

Algorithm

Step 1: Check whether queue is empty.
Step 2: If empty, display "Queue Underflow".
Step 3: Remove the front element.
Step 4: Move Front to the next element.
Step 5: Stop.

Example

Before deletion

10 → 20 → 30 → 40

Delete first element

20 → 30 → 40

Step-by-Step

Before

Front

↓

10 → 20 → 30 → 40

Delete

Remove 10

After

Front

↓

20 → 30 → 40

Real-Time Example

A customer receives a ticket and leaves the queue.

The next customer becomes the first person in the queue.

3. Front() / Peek()

Definition

The **Front()** or **Peek()** operation returns the first element without removing it.

Algorithm

Step 1: Check whether queue is empty.

Step 2: If empty, return NULL.

Step 3: Return Front element.

Step 4: Stop.

Example

Queue

10 → 20 → 30 → 40

Front()

Output

10

The queue remains unchanged.

4. Rear()

Definition

The **Rear()** operation returns the last element of the queue without deleting it.

Example

Queue

10 → 20 → 30 → 40

Rear()

Output

40

5. isEmpty()

Definition

Checks whether the queue contains any elements.

Logic

If Queue Size == 0

Return True

Else

Return False

Example

Queue

[]

Output

True

Example

Queue

10 → 20

Output

False

6. Size()

Definition

Returns the total number of elements present in the queue.

Example

Queue

10 → 20 → 30 → 40

Size()

Output

Queue Operation Summary Table

Operation Description

Enqueue	Insert at Rear
Dequeue	Delete from Front
Front()	View first element
Rear()	View last element
isEmpty()	Check queue is empty
Size()	Count elements

Dry Run Example

Initially

Queue

[]

Enqueue(10)

10

Enqueue(20)

10 → 20

Enqueue(30)

10 → 20 → 30

Front()

Output

10

Queue remains

10 → 20 → 30

Rear()

Output

30

Dequeue()

Remove

10

Queue becomes

20 → 30

Size()

Output

2

isEmpty()

Output

False

Complete Queue Flow

Start

↓

Queue Empty

↓

Enqueue(10)

↓

10

↓

Enqueue(20)

↓

10 → 20

↓

Enqueue(30)

↓

10 → 20 → 30

↓

Front()

↓

10

↓

Rear()

↓

30

↓

Dequeue()

↓

20 → 30

↓

End

Queue Overflow

Definition

Queue Overflow occurs when we try to insert an element into a **full queue**.

Example

Queue Size = 5

Already Stored

10

20

30

40

50

Insert 60

Output

Queue Overflow

Queue Underflow

Definition

Queue Underflow occurs when we try to delete an element from an **empty queue**.

Example

Queue

[]

Delete()

Output

Queue Underflow

TOPIC 4: QUEUES (PART 3)

Queue Implementation using Python (MS Word Friendly | Student Friendly | Exam Ready)

17. Queue Implementation Using Python List

Python provides a built-in **List** that can be used to implement a Queue.

In a Queue:

- Enqueue → `append()`
- Dequeue → `pop(0)`

Although this is simple, `pop(0)` takes **$O(n)$** time because all remaining elements shift one position to the left.

Queue Using Python List

class Queue:

```
def __init__(self):
    self.items = []

# Insert element
def enqueue(self, item):
    self.items.append(item)
    print(item, "inserted into Queue")

# Delete element
def dequeue(self):
```

```

    if self.is_empty():
        print("Queue Underflow")
        return
    return self.items.pop(0)

# View first element
def front(self):
    if self.is_empty():
        print("Queue is Empty")
        return
    return self.items[0]

# View last element
def rear(self):
    if self.is_empty():
        print("Queue is Empty")
        return
    return self.items[-1]

# Check Queue Empty
def is_empty(self):
    return len(self.items) == 0

# Queue Size
def size(self):
    return len(self.items)

# Display Queue
def display(self):

```

```
        print("Queue :", self.items)

q = Queue()

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

q.display()

print("Front :", q.front())
print("Rear :", q.rear())

print("Deleted :", q.dequeue())

q.display()

print("Size :", q.size())
```

Output

```
10 inserted into Queue
20 inserted into Queue
30 inserted into Queue
```

```
Queue : [10, 20, 30]
```

```
Front : 10
```

Rear : 30

Deleted : 10

Queue : [20, 30]

Size : 2

Program Explanation

Step 1

class Queue:

Creates a Queue class.

Step 2

self.items=[]

Creates an empty list.

Step 3

enqueue()

Adds an element at the rear.

Uses

append()

Step 4

dequeue()

Deletes the first element.

Uses

pop(0)

Step 5

front()

Returns the first element.

Step 6

rear()

Returns the last element.

Step 7

size()

Returns total number of elements.

Step 8

display()

Prints all queue elements.

Dry Run

Initially

[]

Enqueue(10)

[10]

Enqueue(20)

[10,20]

Enqueue(30)

[10,20,30]

Dequeue()

Removed : 10

Queue

[20,30]

Front()

20

Rear()

30

Size()

2

Time Complexity (Using List)

Operation Complexity

Enqueue $O(1)$

Dequeue $O(n)$

Front $O(1)$

Rear $O(1)$

Size $O(1)$

Why is pop(0) Slow?

When the first element is removed,
all remaining elements move one position left.

Example

Before

[10,20,30,40]

Delete 10

After

[20,30,40]

20 shifts left.

30 shifts left.

40 shifts left.

Therefore,

Time = $O(n)$

Better Queue Implementation Using deque

Python provides **collections.deque** which is specially designed for Queue operations.

It performs insertion and deletion in **$O(1)$** time.

Queue Using deque

```
from collections import deque
```

```
q = deque()
```

```
q.append(10)
```

```
q.append(20)
```

```
q.append(30)
```

```
print("Queue :", q)
```

```
print("Deleted :", q.popleft())
```

```
print("Queue :", q)
```

```
print("Front :", q[0])
```

```
print("Rear :", q[-1])
```

```
print("Size :", len(q))
```

Output

Queue : deque([10, 20, 30])

Deleted : 10

Queue : deque([20, 30])

Front : 20

Rear : 30

Size : 2

Why deque is Better?

- No shifting of elements.
 - Very fast insertion.
 - Very fast deletion.
 - Suitable for large applications.
 - Used in professional Python programs.
-

List vs deque Comparison

Feature	List	deque
----------------	-------------	--------------

Feature	List	deque
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(n)$	$O(1)$
Insert Front	Slow	Fast
Delete Front	Slow	Fast
Memory Efficiency	Good	Better
Recommended for Queue	No	Yes

Real-Time Example

Suppose customers arrive in this order:

Ravi

Kiran

Anu

Rahul

Queue

Front

↓

Ravi → Kiran → Anu → Rahul

↑

Rear

After serving Ravi,

Front



Kiran → Anu → Rahul



Rear

TOPIC 4: QUEUES (PART 4)

Priority Queue

18. Priority Queue

Definition

A **Priority Queue** is a special type of queue in which each element is assigned a **priority**.

Unlike a normal queue, elements are removed based on **priority**, not just on the order of insertion.

The element with the **highest priority** (or lowest priority, depending on the implementation) is processed first.

Simple Definition

A Priority Queue is a queue in which elements are served according to their priority instead of their arrival time.

19. Real-Life Example

Hospital Emergency

Patients are treated according to the seriousness of their condition.

Priority

1 → Critical Patient

2 → Serious Patient

3 → Normal Patient

Treatment Order

Critical

↓

Serious

↓

Normal

The patient who came first is **not always treated first**.

Another Example

Tasks inside a computer.

Task A → Priority 5

Task B → Priority 1

Task C → Priority 3

Execution Order

Task B

↓

Task C

↓

Task A

Because

$1 < 3 < 5$

20. Difference Between Queue and Priority Queue

Normal Queue	Priority Queue
Follows FIFO	Follows Priority
Arrival decides order	Priority decides order
All elements equal	Every element has priority
Simple structure	Heap / Priority-based structure
Used in ticket counters	Used in CPU scheduling

21. Characteristics of Priority Queue

- Every element has a priority.
 - Highest priority element is removed first.
 - Does not always follow FIFO.
 - Used where urgent tasks must be processed first.
 - Usually implemented using Heap.
 - Efficient for scheduling applications.
-

22. Working of Priority Queue

Suppose we insert

Task A Priority = 4

Task B Priority = 2

Task C Priority = 1

Task D Priority = 5

Queue

Task A (4)

Task B (2)

Task C (1)

Task D (5)

Processing Order

Task C

↓

Task B

↓

Task A

↓

Task D

23. Priority Queue Diagram

Insertion

Task A (4)

Task B (2)

Task C (1)

Task D (5)

↓

Priority Queue

↓

Processing

Task C

↓

Task B

↓

Task A

↓

Task D

24. How Priority Queue Works

Python uses a **Heap (Binary Heap)** to implement a Priority Queue.

The module used is

heapq

Heap automatically keeps the smallest priority element at the top.

25. Python Implementation using heapq

```
import heapq
```

```
pq = []
```

```
heapq.heappush(pq, (2, "Task B"))
```

```
heapq.heappush(pq, (1, "Task A"))
```

```
heapq.heappush(pq, (3, "Task C"))
```

```
print("Priority Queue:")
```

```
print(pq)
```

```
print("\nExecuting Tasks")
```

```
while pq:
```

```
    print(heapq.heappop(pq))
```

Output

Priority Queue

```
[(1,'Task A'),
```

```
 (2,'Task B'),
```

(3,'Task C')]

Executing Tasks

(1,'Task A')

(2,'Task B')

(3,'Task C')

Program Explanation

Step 1

```
import heapq
```

Imports the Heap module.

Step 2

```
pq=[]
```

Creates an empty priority queue.

Step 3

```
heappush()
```

Adds an element while maintaining priority order.

Step 4

```
heappop()
```

Removes the element having the highest priority (smallest number).

Step 5

Loop continues until queue becomes empty.

Dry Run

Initially

[]

Insert

(2,B)

[(2,B)]

Insert

(1,A)

[(1,A),(2,B)]

Insert

(3,C)

[(1,A),(2,B),(3,C)]

Remove

(1,A)

Remaining

[(2,B),(3,C)]

Time Complexity

Operation	Complexity
-----------	------------

Insert	$O(\log n)$
--------	-------------

Delete Highest Priority	$O(\log n)$
-------------------------	-------------

Peek	$O(1)$
------	--------

Build Heap	$O(n)$
------------	--------

Advantages of Priority Queue

- Faster task scheduling.
- Processes important tasks first.
- Efficient Heap implementation.

- Used in Operating Systems.
 - Useful in Networking.
 - Reduces waiting time for urgent jobs.
 - Suitable for real-time applications.
-

Disadvantages

- More complex than a normal queue.
 - Heap implementation requires extra logic.
 - Equal priorities may need additional handling.
 - Slightly more memory usage.
-

Applications of Normal Queue

- CPU Scheduling (Round Robin)
 - Printer Queue
 - Ticket Reservation
 - BFS (Breadth First Search)
 - Network Packet Handling
 - Call Centers
 - Order Processing
 - I/O Buffers
-

Applications of Priority Queue

- CPU Priority Scheduling
- Dijkstra's Shortest Path Algorithm
- A* Search Algorithm
- Huffman Coding
- Event Simulation
- Load Balancing
- Emergency Hospital Systems

- Airline Reservation Systems
 - Task Scheduling
 - Operating Systems
-

Case Study 1 – Printer Queue

Problem

Many users send print jobs.

Printing should happen in the order they are submitted.

Solution

User1

↓

User2

↓

User3

↓

Printer

FIFO is followed.

Case Study 2 – CPU Scheduling

Processes

P1 Priority = 4

P2 Priority = 1

P3 Priority = 3

Execution Order

P2

↓

P3

↓

P1

CPU always selects the highest priority process first.

Case Study 3 – GPS Navigation (Dijkstra Algorithm)

Suppose GPS finds three roads.

Road A Distance = 10 km

Road B Distance = 5 km

Road C Distance = 8 km

Priority Queue always selects

Road B

↓

Road C

↓

Road A

Shortest path is processed first.

UNIT-V: CHAPTER-5

GUI Programming with Tkinter (Part – 1)

Introduction, Tkinter Basics, Architecture, Components, Creating First GUI

GUI Programming with Tkinter

Introduction

GUI (Graphical User Interface) is a way of interacting with a computer using graphical components such as windows, buttons, menus, icons, text boxes, and images instead of typing commands.

Python provides several libraries for GUI development. Among them, **Tkinter** is the most popular and beginner-friendly library.

Tkinter is included with the standard Python installation, so no separate installation is required in most cases.

What is GUI?

A **Graphical User Interface (GUI)** is an interface that allows users to interact with software through graphical elements instead of text-based commands.

Examples include:

- Calculator
- Notepad
- Login Form
- ATM Screen
- File Explorer
- Student Management System
- Banking Software

Definition of GUI

GUI (Graphical User Interface) is a visual interface that enables users to interact with applications using graphical controls such as buttons, labels, text boxes, menus, and images.

What is Tkinter?

Tkinter is Python's standard GUI (Graphical User Interface) library.

It is a thin object-oriented layer built on top of the Tcl/Tk toolkit.

Tkinter allows developers to create desktop applications quickly using simple Python code.

It provides:

- Windows
- Buttons
- Labels
- Text Boxes
- Menus
- Canvas
- Frames
- Event Handling
- Layout Managers

Definition of Tkinter

Tkinter is the standard GUI library in Python used to create desktop applications using graphical widgets and event handling.

Why do we use Tkinter?

Tkinter is used because it:

- Creates graphical applications easily.
- Is included with Python.
- Is easy for beginners.

- Supports event-driven programming.
 - Runs on Windows, Linux, and macOS.
 - Provides many built-in widgets.
-

Features of Tkinter

1. Built into Python.
 2. Object-Oriented.
 3. Cross-platform support.
 4. Simple syntax.
 5. Event-driven programming.
 6. Rich collection of widgets.
 7. Multiple layout managers.
 8. Supports graphics using Canvas.
 9. Supports menus.
 10. Easy debugging.
 11. Lightweight.
 12. Fast application development.
-

Real-Time Applications of Tkinter

Tkinter is used to develop:

- Calculator
- Login Form
- Student Management System
- Library Management System
- Banking Application
- Attendance System
- Quiz Application
- Text Editor
- Inventory Software

- Billing Software

Real-Life Example

Suppose you open an ATM.

You do not type commands.

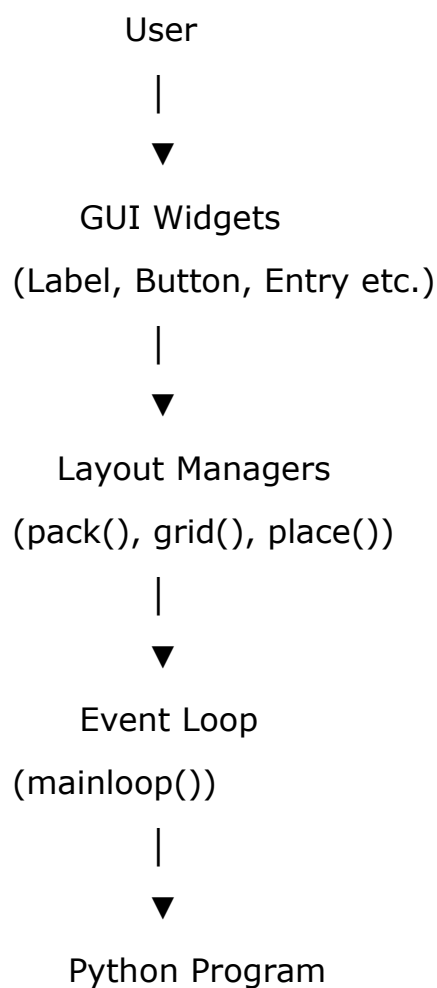
Instead, you press buttons displayed on the screen.

Those buttons are GUI components.

Similarly, a Tkinter application allows users to interact using buttons, text boxes, and menus.

Tkinter Architecture

The textbook explains that a Tkinter program mainly consists of:



Basic Flow of a Tkinter Program

Every Tkinter application follows the same sequence.

Import Tkinter



Create Root Window



Create Widgets



Arrange Widgets



Handle Events



Start mainloop()

Components of Tkinter

According to the textbook, the main components are:

1. Tk()

Creates the main application window.

Example:

```
root = Tk()
```

2. Widgets

Widgets are the visible graphical components placed on the window.

Examples:

- Label
- Button
- Entry
- Text
- Listbox
- Canvas

- Menu

3. Geometry Managers

Geometry managers decide where widgets appear.

Tkinter provides:

- `pack()`
- `grid()`
- `place()`

These will be discussed in Part-3.

4. `mainloop()`

Keeps the application running.

It continuously waits for user actions like:

- Mouse Click
- Keyboard Input
- Button Click

Without `mainloop()`, the window closes immediately.

Importing Tkinter

There are different ways to import Tkinter.

Method 1 (Most Common)

```
import tkinter as tk
```

Example:

```
import tkinter as tk
```

```
root = tk.Tk()
```

```
root.mainloop()
```

Method 2

```
from tkinter import *
```

Example:

```
from tkinter import *
```

```
root = Tk()
```

```
mainloop()
```

Creating the First GUI Window

Program:

```
import tkinter as tk
```

```
root = tk.Tk()
```

```
root.title("My First GUI")
```

```
root.geometry("300x200")
```

```
root.mainloop()
```

Program Explanation

```
import tkinter as tk
```

Imports the Tkinter library.

```
root = tk.Tk()
```

Creates the main window.

```
root.title("My First GUI")
```

Sets the window title.

```
root.geometry("300x200")
```

Sets the window size.

Width = 300 pixels

Height = 200 pixels

```
root.mainloop()
```

Starts the event loop.

The program keeps running until the user closes the window.

Output

```
+-----+
|  My First GUI  |
|                |
|                |
|                |
|                |
|                |
+-----+
```

What is the Root Window?

The root window is the first and main window of every Tkinter application.

Every widget is placed inside this window.

Example:

Root Window

- └─ Label
 - └─ Button
 - └─ Entry
 - └─ Menu
 - └─ Canvas
-

Why is `mainloop()` Important?

The textbook mentions that `mainloop()` keeps listening for user interaction.

Without it:

- Window opens.
- Window closes immediately.

With it:

- Window remains active.
- Accepts keyboard and mouse events.
- Responds to user actions.

Execution Flow of a Tkinter Program

Program Starts

|



Import Tkinter

|



Create Tk()

|



Create Widgets

|



Arrange Widgets

|



`mainloop()`

|



Wait for User Events

|



Execute Callback Function

Advantages of Tkinter

- Free and open source.
 - Included with Python.
 - Easy to learn.
 - Cross-platform.
 - Lightweight.
 - Supports event-driven programming.
 - Rich collection of widgets.
 - Suitable for educational and desktop applications.
-

Limitations of Tkinter

- Limited modern UI design.
 - Less suitable for advanced animations.
 - Fewer built-in themes compared to modern GUI frameworks.
 - Not ideal for large enterprise GUI applications.
-

UNIT-V : CHAPTER-5

GUI Programming with Tkinter (Part – 2)

Common Widgets – Label, Button, Entry, Text, Menu, Listbox, Canvas, Checkbutton, Radiobutton, Frame

Common Widgets in Tkinter

What is a Widget?

A **Widget** is a graphical component used to build a GUI application.

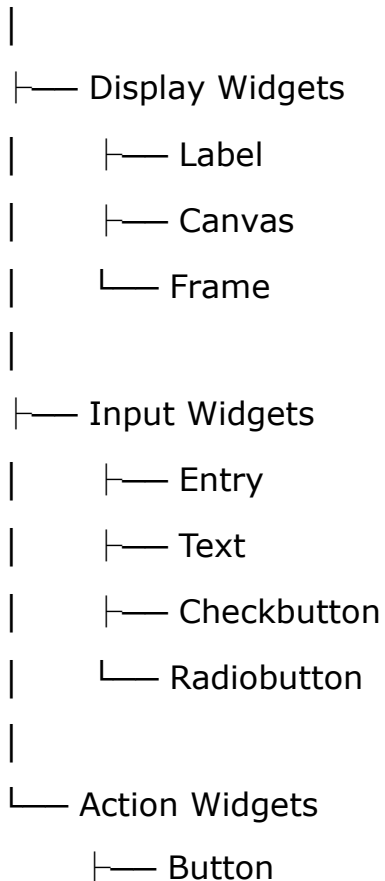
Examples:

- Label
- Button
- Entry
- Text
- Menu
- Listbox
- Canvas
- Checkbutton
- Radiobutton
- Frame

Widgets help users interact with the application.

Classification of Widgets

Tkinter Widgets



└─ Menu

└─ Listbox

1. Label Widget

Definition

A **Label** is used to display text or images.

It is used only for displaying information.

The user cannot edit a Label.

Purpose

Display

- Titles
- Headings
- Instructions
- Images
- Status Messages

Constructor

```
tk.Label(parent, text="", font=(), fg="", bg="")
```

Common Options

Option Meaning

text Display text

font Font style

fg Text color

bg Background color

width Width

height Height

Option Meaning

image Display image

justify Text alignment

Important Methods

config()

cget()

Program

```
import tkinter as tk
```

```
root = tk.Tk()
```

```
label = tk.Label(root,  
                  text="Welcome to Python",  
                  font=("Arial",16),  
                  fg="blue")
```

```
label.pack()
```

```
root.mainloop()
```

Output

```
-----  
Welcome to Python  
-----
```

Real-Time Example

College Name

Hospital Name

ATM Instructions

Application Heading

2. Button Widget

Definition

A **Button** is used to perform an action when clicked.

Purpose

Execute

- Submit
 - Save
 - Delete
 - Search
 - Login
 - Exit
-

Constructor

`tk.Button(parent,text="",command=function)`

Common Options

Option	Meaning
--------	---------

text	Button Name
------	-------------

command	Function to execute
---------	---------------------

state	Enable/Disable
-------	----------------

width	Width
-------	-------

height	Height
--------	--------

Option	Meaning
---------------	----------------

bg	Background Color
----	------------------

fg	Foreground Color
----	------------------

Important Methods

invoke()

config()

Program

```
import tkinter as tk
```

```
def message():
```

```
    print("Button Clicked")
```

```
root=tk.Tk()
```

```
btn=tk.Button(root,
```

```
    text="Click Me",
```

```
    command=message)
```

```
btn.pack()
```

```
root.mainloop()
```

Output

Click Me

After clicking

Button Clicked

Real-Time Examples

Login Button

Submit Button

Save Button

Print Button

Register Button

3. Entry Widget

Definition

Entry widget accepts **single-line user input**.

Purpose

Receive

- Name
 - Age
 - Username
 - Password
 - Mobile Number
-

Constructor

tk.Entry(parent)

Common Options

Option	Meaning
width	Entry Width
textvariable	Variable
show	Hide Password

Option	Meaning
---------------	----------------

font	Font
------	------

Important Methods

get()

insert()

delete()

Program

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
entry=tk.Entry(root,width=25)
```

```
entry.pack()
```

```
root.mainloop()
```

Password Example

```
password=tk.Entry(root,show="*")
```

Real-Time Examples

Login Form

Registration Form

Student Form

Bank Form

Difference Between Label and Entry

Label	Entry
Displays Text	Accepts Input
Read Only	Editable
Output Widget	Input Widget

4. Text Widget

Definition

Text widget accepts **multiple lines of input**.

Purpose

Used for

- Feedback
 - Address
 - Description
 - Comments
-

Constructor

`tk.Text(parent)`

Important Methods

`insert()`

`get()`

`delete()`

Program

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
text=tk.Text(root,  
              width=30,  
              height=5)
```

```
text.pack()
```

```
root.mainloop()
```

Real-Time Example

Student Address

Feedback Form

Essay Writing

Complaint Form

Difference Between Entry and Text

Entry	Text
--------------	-------------

Single Line	Multiple Lines
-------------	----------------

Small Input	Large Input
-------------	-------------

5. Menu Widget

Definition

A Menu provides a list of commands.

Purpose

Create

File
Edit
View
Help

Constructor

Menu(parent)

Common Methods

add_command()

add_separator()

add_cascade()

Program

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
menubar=tk.Menu(root)
```

```
file=tk.Menu(menubar)
```

```
file.add_command(label="Open")
```

```
file.add_command(label="Save")
```

```
file.add_separator()
```

```
file.add_command(label="Exit")
```

```
menubar.add_cascade(label="File",menu=file)
```

```
root.config(menu=menubar)
```

```
root.mainloop()
```

Output

File

Open

Save

Exit

Real-Time Example

MS Word

Notepad

Paint

Browser

6. Listbox Widget

Definition

Listbox displays a list of items.

The user can select one or more items.

Purpose

Display
Course List
City List
Subject List
Employee List

Constructor

tk.Listbox(parent)

Important Methods

insert()

delete()

curselection()

Program

```
import tkinter as tk  
  
root=tk.Tk()  
  
listbox=tk.Listbox(root)  
  
listbox.insert(1,"Python")  
  
listbox.insert(2,"Java")  
  
listbox.insert(3,"C")
```

```
listbox.pack()
```

```
root.mainloop()
```

Output

Python

Java

C

7. Canvas Widget

Definition

Canvas is used to draw graphics.

Purpose

Draw

Lines

Circles

Rectangles

Polygons

Images

Charts

Constructor

Canvas(parent)

Important Methods

`create_line()`

`create_rectangle()`

`create_oval()`

`create_polygon()`

Program

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
canvas=tk.Canvas(root,  
                  width=250,  
                  height=200)
```

```
canvas.create_rectangle(50,50,150,100)
```

```
canvas.pack()
```

```
root.mainloop()
```

Output

Rectangle appears on the window.

Real-Time Examples

Paint Application

Drawing Tool

Charts

Games

8. Checkbutton Widget

Definition

A Checkbutton allows users to select multiple options.

Purpose

Multiple Choice Selection

Examples

Skills

Languages

Hobbies

Subjects

Constructor

`Checkbutton(parent,text="")`

Program

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
python=tk.Checkbutton(root,text="Python")
```

```
java=tk.Checkbutton(root,text="Java")
```

```
python.pack()
```

```
java.pack()
```

```
root.mainloop()
```

Output

☐ Python

☐ Java

9. Radiobutton Widget

Definition

Radiobutton allows users to select **only one option**.

Purpose

Single Choice Selection

Examples

Gender

Payment Mode

Semester

Branch

Constructor

```
Radiobutton(parent,text="")
```

Program

```
import tkinter as tk
```

```

root=tk.Tk()

gender=tk.StringVar()

tk.Radiobutton(root,text="Male",
               variable=gender,
               value="Male").pack()

tk.Radiobutton(root,text="Female",
               variable=gender,
               value="Female").pack()

root.mainloop()

```

Output

- Male

- Female

Only one option can be selected.

Difference Between Checkbutton and Radiobutton

Checkbutton	Radiobutton
Multiple Selection	Single Selection
Checkbox	Radio Circle
Independent	Linked using Variable

10. Frame Widget

Definition

A Frame is a container widget used to group related widgets.

Purpose

Organize GUI

Group Buttons

Group Labels

Group Forms

Constructor

Frame(parent)

Program

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
frame=tk.Frame(root,  
                bd=2,  
                relief="solid")
```

```
frame.pack()
```

```
tk.Label(frame,text="Name").pack()
```

```
tk.Entry(frame).pack()
```

```
root.mainloop()
```

Output

A bordered frame containing a Label and an Entry widget.

Summary Table

Widget	Purpose
Label	Display Text/Image
Button	Perform Action
Entry	Single Line Input
Text	Multiple Line Input
Menu	Menu Commands
Listbox	Display List
Canvas	Drawing Graphics
Checkbutton	Multiple Selection
Radiobutton	Single Selection
Frame	Group Widgets

UNIT-V : CHAPTER-5

GUI Programming with Tkinter (Part -3)

Layout Managers, Event Handling, Command Callback, Event Binding, Event Propagation

3. Layout Managers in Tkinter

Definition

A **Layout Manager** is used to arrange widgets inside a window.

Without a layout manager, widgets cannot be positioned properly.

Tkinter provides **three layout managers**:

1. pack()
 2. grid()
 3. place()
-

Comparison of Layout Managers

Layout Manager	Purpose	Best Used For
pack()	Arranges widgets vertically or horizontally	Simple GUIs
grid()	Arranges widgets in rows and columns	Login forms, Registration forms
place()	Places widgets using exact coordinates	Custom layouts

1. pack() Layout Manager

Definition

The **pack()** manager arranges widgets automatically.

Widgets can be placed:

- Top
 - Bottom
 - Left
 - Right
-

Syntax

```
widget.pack()
```

Common Options

side

fill

expand

padx

pady

Example

```
import tkinter as tk
```

```
root = tk.Tk()
```

```
tk.Label(root,text="Python").pack()
```

```
tk.Button(root,text="Submit").pack()
```

```
root.mainloop()
```

Output

Python

Submit

Advantages

- Easy to use
 - Best for beginners
 - Suitable for small applications
-

Disadvantages

- Difficult for complex layouts
 - Less control over widget position
-

2. grid() Layout Manager

Definition

The **grid()** manager arranges widgets in rows and columns.

Each widget is placed using row and column numbers.

Syntax

```
widget.grid(row=0,column=0)
```

Common Options

row

column

padx

pady

sticky

Example

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
tk.Label(root,text="Username").grid(row=0,column=0)
```

```
tk.Entry(root).grid(row=0,column=1)
```

```
tk.Label(root,text="Password").grid(row=1,column=0)
```

```
tk.Entry(root,show="*").grid(row=1,column=1)
```

```
root.mainloop()
```

Output

Username [_____]

Password [_____]

Advantages

- Clean arrangement
 - Best for forms
 - Easy to understand
-

Disadvantages

- Slightly more coding
-

3. place() Layout Manager

Definition

The **place()** manager positions widgets using exact coordinates.

Syntax

```
widget.place(x=50,y=100)
```

Example

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
tk.Label(root,text="Python").place(x=50,y=40)
```

```
tk.Button(root,text="OK").place(x=70,y=90)
```

```
root.mainloop()
```

Output

Widgets appear exactly at the specified coordinates.

Advantages

- Full control over position
 - Useful for custom interfaces
-

Disadvantages

- Not responsive
 - Position changes on different screen sizes
-

Comparison of pack(), grid(), and place()

Feature	pack()	grid()	place()
Arrangement	Automatic	Rows & Columns	Coordinates
Easy to Learn	Yes	Yes	Moderate
Best For	Small Apps	Forms	Custom Layouts
Responsive	Yes	Yes	No

Important Rule

Do **NOT** use **pack()** and **grid()** inside the same Frame.

Example:

✗ Wrong

```
label.pack()
```

```
button.grid(row=0,column=0)
```

✓ Correct

Use only one layout manager inside one container.

Event Handling in Tkinter

Definition

An **Event** is an action performed by the user.

Examples:

- Mouse Click
 - Keyboard Press
 - Mouse Movement
 - Double Click
 - Window Close
-

What is Event Handling?

Event Handling is the process of detecting a user action and executing the corresponding function.

Event Flow

User Action

↓

Event Generated

↓

Event Handler Executes

↓

GUI Updated

Real-Time Examples of Events

User Action Event

Button Click Mouse Event

Typing Keyboard Event

Mouse Move Motion Event

Double Click Double Button Event

Window Close Close Event

Event Loop

Definition

The **Event Loop** continuously waits for user actions.

It is started using:

```
root.mainloop()
```

Working of Event Loop

Start Program

↓

Create Window

↓

```
mainloop()
```

↓

Wait for User Action

↓

Process Event

↓

Update GUI

↓

Wait Again

Command Callback (High-Level Event)

Definition

A **Command Callback** is used with widgets like:

- Button
- Menu
- Checkbutton
- Radiobutton

When the widget is clicked, the associated function is executed automatically.

Syntax

```
Button(root,  
        text="Click",  
        command=myFunction)
```

Example

```
import tkinter as tk

def hello():
    print("Welcome")

root=tk.Tk()

btn=tk.Button(root,
               text="Click",
               command=hello)

btn.pack()

root.mainloop()
```

Output

Welcome
(after clicking the button)

Features of Command Callback

- Very simple
- No event parameter
- Best for buttons
- Easy for beginners

Event Binding (bind Method)

Definition

The **bind()** method connects a specific event with a function.

It is used for:

- Keyboard Events
- Mouse Events
- Double Click
- Mouse Movement

Syntax

```
widget.bind("<Event>", handler)
```

Example

```
import tkinter as tk

def key_pressed(event):
    print("Key:", event.keysym)

root=tk.Tk()

root.bind("<Key>", key_pressed)

root.mainloop()
```

Output

Key : A

Key : B

Key : C

(as the user presses keys)

Difference Between Command and bind()

Command	bind()
---------	--------

Used for Buttons	Used for Any Event
------------------	--------------------

No Event Object	Receives Event Object
-----------------	-----------------------

Simple	More Powerful
--------	---------------

High-Level Event	Low-Level Event
------------------	-----------------

Common Event Sequences

Event	Meaning
<Button-1>	Left Mouse Click
<Button-3>	Right Mouse Click
<Double-Button-1>	Double Click
<Motion>	Mouse Move
<Enter>	Mouse Enters Widget
<Leave>	Mouse Leaves Widget
<KeyPress>	Key Pressed
<KeyRelease>	Key Released
<Return>	Enter Key Pressed

Event Object

The **Event Object** stores information about the event.

Examples:

event.x

event.y

event.keysym

event.char

Example

```
def click(event):
```

```
    print(event.x)
```

```
    print(event.y)
```

Real-Time Applications of Event Handling

- Login Form
 - Calculator
 - Quiz Application
 - Paint Program
 - Drawing Application
 - Text Editor
 - ATM Software
 - Banking Software
 - Student Management System
-

UNIT-V : CHAPTER-5

GUI Programming with Tkinter (Part -4)

Advantages, Disadvantages, Applications, Building GUI Applications, OOP Design, Complete Examples

Advantages of Event Handling in Tkinter

Event handling makes GUI applications interactive and user-friendly.

1. Simple and Beginner Friendly

Tkinter uses simple syntax for handling events.

Example:

```
button = tk.Button(root, text="Submit", command=submit)
```

Only one line is enough to connect a button with a function.

2. Flexible

Tkinter supports both

- High-level events (command)
- Low-level events (bind)

Examples

Button Click

Keyboard Press

Mouse Click

Mouse Move

Double Click

3. Built-in Event Object

The event object stores information about the user's action.

Example

event.x

event.y

event.char

event.keysym

4. Modular Programming

Each event can have its own function.

Example

```
def save():
```

```
    pass
```

```
def delete():
```

```
    pass
```

```
def update():
```

```
    pass
```

This makes the program organized and easy to maintain.

5. Cross Platform

Tkinter applications work on

- Windows
- Linux
- macOS

without changing the code.

Disadvantages of Event Handling

Although Tkinter is easy, it also has some limitations.

1. Limited Advanced GUI Features

Tkinter is suitable for simple desktop applications.

It is not ideal for

- Advanced animations
- Multimedia applications
- Modern UI effects

2. Performance Issues

If many events occur continuously,
GUI performance becomes slower.

Example

Mouse movement every millisecond
creates thousands of events.

3. Difficult Debugging

Large applications contain many event handlers.
Finding the source of an error becomes difficult.

4. Older User Interface

Tkinter provides a simple interface.

Compared to

PyQt

Kivy

Flutter

the appearance looks older.

Applications of Tkinter Event Handling

Tkinter is used in many desktop applications.

Examples

- Login Systems
- Student Management Systems
- Library Management
- Employee Management
- Banking Software
- Billing Software

- Quiz Applications
 - Calculator
 - Paint Program
 - Drawing Applications
 - Attendance System
 - Inventory System
 - Hospital Management
 - Notepad
 - File Explorer
 - To-Do List
 - Reminder Software
-

Building Simple GUI Applications

The textbook explains a standard design process.

Follow this sequence.

Step 1

Import Tkinter

```
import tkinter as tk
```

Step 2

Create Root Window

```
root = tk.Tk()
```

Step 3

Set Window Title

```
root.title("Student Management")
```

Step 4

Set Window Size

```
root.geometry("400x300")
```

Step 5

Create Widgets

Examples

Label

Button

Entry

Listbox

Menu

Step 6

Arrange Widgets

Using

pack()

grid()

place()

Step 7

Write Functions

Example

```
def submit():
```

```
print("Submitted")
```

Step 8

Connect Widgets

```
Button(root,  
text="Submit",  
command=submit)
```

Step 9

Start Event Loop

```
root.mainloop()
```

GUI Development Flow

Import Tkinter

↓

Create Root Window

↓

Create Widgets

↓

Arrange Widgets

↓

Write Functions



Connect Events



mainloop()



GUI Ready

OOP-Based GUI Design

Instead of writing everything together,
we create a class.

Why OOP?

Advantages

- Organized code
- Easy maintenance
- Reusable code
- Better readability
- Easy debugging

Basic Structure

class MyApp:

```
def __init__(self,root):
```

```
    self.root=root
```

Example

```
import tkinter as tk
```

```
class MyApp:
```

```
    def __init__(self,root):
```

```
        self.root=root
```

```
        self.root.title("Demo")
```

```
        tk.Label(root,text="Welcome").pack()
```

```
root=tk.Tk()
```

```
obj=MyApp(root)
```

```
root.mainloop()
```

Real-Time Example

Student Management System

Student Name

Roll Number

Course

Submit Button



Save Student Details

Example 1

Simple Login Form

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
root.title("Login")
```

```
tk.Label(root,text="Username").grid(row=0,column=0)
```

```
tk.Entry(root).grid(row=0,column=1)
```

```
tk.Label(root,text="Password").grid(row=1,column=0)
```

```
tk.Entry(root,show="*").grid(row=1,column=1)
```

```
tk.Button(root,text="Login").grid(row=2,column=1)
```

```
root.mainloop()
```

Output

Username _____

Password _____

Login

Example 2

Student Registration Form

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
tk.Label(root,text="Name").pack()
```

```
tk.Entry(root).pack()
```

```
tk.Label(root,text="Age").pack()
```

```
tk.Entry(root).pack()
```

```
tk.Button(root,text="Register").pack()
```

```
root.mainloop()
```

Example 3

Calculator Window

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
tk.Label(root,text="Calculator").pack()
```

```
tk.Entry(root).pack()
```

```
tk.Button(root,text="Calculate").pack()
```

```
root.mainloop()
```

Form Validation Example

The textbook explains validating user input before processing.

Example

```
from tkinter import messagebox
```

```
def submit():
```

```
    if name.get()=="":
```

```
        messagebox.showerror("Error","Name Required")
```

Validation prevents invalid data.

To-Do List Using Listbox

```
import tkinter as tk
```

```
root=tk.Tk()
```

```
lb=tk.Listbox(root)
```

```
lb.pack()
```

```
lb.insert(1,"Python")
```

```
lb.insert(2,"Java")
```

```
lb.insert(3,"C Language")
```

```
root.mainloop()
```

Output

Python

Java

C Language

Best Practices While Developing GUI

- Keep interface simple.
 - Use meaningful button names.
 - Validate user input.
 - Use functions.
 - Use classes for large projects.
 - Keep code modular.
 - Use proper spacing.
 - Handle errors.
-

Real-Time Applications

Education

- Quiz System

- Attendance
- Online Test

Business

- Billing Software
- Inventory
- Employee Management

Medical

- Hospital Management
- Patient Registration

Banking

- ATM Interface
- Customer Registration

College

- Student Result System
- Library System
- Timetable Generator

Mini Project Ideas

1. Calculator
 2. Login Form
 3. Student Registration
 4. Employee Management
 5. Quiz Application
 6. Library Management
 7. Attendance System
 8. Currency Converter
 9. To-Do List
 10. Student Database GUI
-

UNIT-V : CHAPTER-5

GUI Programming with Tkinter (Part – 5)

Additional Topics: collections.deque, Searching Algorithms, Sorting Algorithms, Comparison Tables & Revision

1. collections.deque

Definition

A **deque (Double Ended Queue)** is a data structure provided by the **collections** module in Python.

It allows insertion and deletion of elements from **both the front and rear** efficiently.

Unlike a normal Python list, deque performs these operations in **O(1)** time.

Why do we use deque?

A normal list is slow when removing the first element because all remaining elements shift left.

A deque avoids this problem.

Import Statement

```
from collections import deque
```

Creating a deque

```
from collections import deque
```

```
q = deque()
```

Important Methods

Method	Purpose
--------	---------

append()	Insert at rear
----------	----------------

Method	Purpose
appendleft()	Insert at front
pop()	Delete from rear
popleft()	Delete from front
clear()	Remove all elements
rotate()	Rotate elements
extend()	Add multiple elements

Program

```
from collections import deque
```

```
q = deque()
```

```
q.append("Python")
```

```
q.append("Java")
```

```
q.appendleft("C")
```

```
print(q)
```

```
print(q.popleft())
```

```
print(q)
```

Output

```
deque(['C', 'Python', 'Java'])
```

```
C
```

```
deque(['Python', 'Java'])
```

Real-Time Applications

- Browser History
 - Undo / Redo
 - Queue Processing
 - BFS Algorithm
 - Sliding Window Problems
-

2. Linear Search

Definition

Linear Search searches each element one by one until the required element is found.

It is the simplest searching algorithm.

Algorithm

1. Start from the first element.
 2. Compare with the key.
 3. If found, stop.
 4. Otherwise move to the next element.
 5. Repeat until the element is found or the list ends.
-

Flow

Start

|



First Element

|

Compare

|

Found?

|— Yes → Stop

|— No

|

Next Element

|

Repeat

Python Program

```
numbers = [10,20,30,40,50]
```

```
key = 30
```

```
for i in range(len(numbers)):
```

```
    if numbers[i] == key:
```

```
        print("Element Found at Position", i+1)
```

```
        break
```

Output

Element Found at Position 3

Time Complexity

Best Case : **O(1)**

Worst Case : **O(n)**

Advantages

- Very simple
- Works on sorted and unsorted data

- Easy implementation

Disadvantages

- Slow for large data
 - Checks every element
-

3. Binary Search

Definition

Binary Search searches an element by repeatedly dividing a **sorted list** into two halves.

Important Note

Binary Search works only on sorted data.

Working

Example

10 20 30 40 50 60 70

Search = 50

↓

Middle = 40

↓

50 > 40

↓

Search Right Half

↓

Found

Python Program

```
numbers=[10,20,30,40,50,60]
```

```
key=50
```

```
low=0
```

```
high=len(numbers)-1
```

```
while low<=high:
```

```
    mid=(low+high)//2
```

```
    if numbers[mid]==key:
```

```
        print("Element Found")
```

```
        break
```

```
    elif numbers[mid]<key:
```

```
        low=mid+1
```

```
    else:
```

high=mid-1

Output

Element Found

Time Complexity

Best Case : **$O(1)$**

Worst Case : **$O(\log n)$**

Advantages

- Faster than Linear Search
 - Efficient for large data
-

Disadvantages

- Requires sorted data
 - More complex than Linear Search
-

Difference Between Linear Search and Binary Search

Linear Search	Binary Search
---------------	---------------

Works on any list	Works only on sorted list
-------------------	---------------------------

$O(n)$	$O(\log n)$
--------	-------------

Easy	Moderate
------	----------

Sequential Search	Divide and Conquer
-------------------	--------------------

4. Bubble Sort

Definition

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Example

Before

40 20 30 10

After

10 20 30 40

Python Program

```
numbers=[40,20,30,10]
```

```
n=len(numbers)
```

```
for i in range(n):
```

```
    for j in range(n-i-1):
```

```
        if numbers[j]>numbers[j+1]:
```

```
            numbers[j],numbers[j+1]=numbers[j+1],numbers[j]
```

```
print(numbers)
```

Output

```
[10, 20, 30, 40]
```

Time Complexity

Best : **$O(n)$**

Worst : **$O(n^2)$**

5. Quick Sort

Definition

Quick Sort is a **Divide and Conquer** sorting algorithm.

It selects a **Pivot** element and partitions the list.

Steps

1. Select Pivot.
 2. Divide list.
 3. Sort Left.
 4. Sort Right.
 5. Combine.
-

Python Program

```
def quick_sort(arr):

    if len(arr)<=1:
        return arr

    pivot=arr[len(arr)//2]

    left=[x for x in arr if x<pivot]

    middle=[x for x in arr if x==pivot]

    right=[x for x in arr if x>pivot]

    return quick_sort(left)+middle+quick_sort(right)

numbers=[40,10,30,20]
```

```
print(quick_sort(numbers))
```

Output

[10, 20, 30, 40]

Time Complexity

Best : **$O(n \log n)$**

Average : **$O(n \log n)$**

Worst : **$O(n^2)$**

6. Merge Sort

Definition

Merge Sort is also a **Divide and Conquer** algorithm.

It divides the list into halves, sorts each half, and merges them.

Working

40 20 10 30

↓

40 20

10 30

↓

40

20

10

30

↓

Merge

↓

10 20 30 40

Python Program

```
def merge_sort(arr):
```

```
    if len(arr)>1:
```

```
        mid=len(arr)//2
```

```
        left=arr[:mid]
```

```
        right=arr[mid:]
```

```
        merge_sort(left)
```

```
        merge_sort(right)
```

```
i=j=k=0
```

```
while i<len(left) and j<len(right):
```

```
    if left[i]<right[j]:
```

```
        arr[k]=left[i]
```

```
        i+=1
```

```
    else:
```

```
        arr[k]=right[j]
```

```
        j+=1
```

```
    k+=1
```

```
while i<len(left):
```

```
    arr[k]=left[i]
```

```
    i+=1
```

```
    k+=1
```

```
while j<len(right):
```

```
    arr[k]=right[j]
```

```
j+=1
```

```
k+=1
```

```
numbers=[40,20,10,30]
```

```
merge_sort(numbers)
```

```
print(numbers)
```

Output

```
[10, 20, 30, 40]
```

Time Complexity

Best : **$O(n \log n)$**

Worst : **$O(n \log n)$**

Comparison of Sorting Algorithms

Feature	Bubble	Quick	Merge
Best Case	$O(n)$	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n \log n)$
Stable	Yes	No	Yes
Faster	No	Yes	Yes

UNIT – V : CHAPTER – 5

GUI Programming with Tkinter (Part – 6)

Additional Notes, Common Errors, Best Practices, Revision & Lab Programs

1. Widget Life Cycle

Definition

Every widget in Tkinter follows a sequence of steps from creation to destruction. This sequence is called the **Widget Life Cycle**.

Life Cycle Diagram

Create Widget

|



Configure Widget

|



Place Widget

(pack / grid / place)

|



Display on Window

|



Wait for User Event

|



Execute Function

|



Update Widget

|



Destroy Widget (Optional)

Explanation

Create Widget

The widget object is created.



Configure Widget

Properties like text, color and font are assigned.



Place Widget

The widget is arranged using pack(), grid() or place().



Wait for Event

The program waits for the user.



Execute Function

The required function executes.



Update GUI

The interface is refreshed.

2. Widget Hierarchy

Every Tkinter application follows a hierarchy.

Tk()



└─ Frame

| └─ Label

| └─ Entry

| └─ Button

| └─ Checkbutton

|

└─ Menu

└─ Canvas

└─ Listbox

└─ Text

3. Naming Convention

Use meaningful variable names.

Good Practice Avoid

lbl_name a

btn_save b

txt_username c

txt_password d

frame_login x

4. Frequently Used Widget Methods

Method Purpose

pack()	Arrange widget
grid()	Arrange widget
place()	Arrange widget
config()	Change properties
get()	Read value
insert()	Insert data
delete()	Remove data
destroy()	Close widget
focus()	Set cursor

5. Common Errors in Tkinter

Error 1

Forgetting mainloop()

Wrong

```
root = Tk()
```

Correct

```
root = Tk()
```

```
root.mainloop()
```

Error 2

Using pack() and grid() together

Wrong

```
label.pack()
```

```
button.grid(row=0,column=0)
```

Use only one layout manager inside the same parent.

Error 3

Wrong command

Wrong

command=hello()

Correct

command=hello

Error 4

Forgetting import statement

Wrong

root=Tk()

Correct

from tkinter import *

root=Tk()

Error 5

Incorrect geometry

Wrong

geometry(300,200)

Correct

geometry("300x200")

6. GUI Design Tips

- Keep the interface simple.
- Use meaningful labels.
- Use readable fonts.
- Align widgets properly.

- Validate user input.
 - Use Frames to organize widgets.
 - Keep buttons at the bottom.
 - Use suitable colors.
 - Write separate functions.
 - Follow proper naming conventions.
-

7. Best Practices

- Write modular code.
 - Avoid duplicate code.
 - Use functions.
 - Use classes for large applications.
 - Handle exceptions.
 - Validate user input.
 - Comment important code.
 - Keep GUI responsive.
-

8. One Page Revision

GUI

Graphical User Interface

Tkinter

Python GUI Library

Main Window

Tk()

Event Loop

mainloop()

Layout Managers

pack()

grid()

place()

Widgets

Label

Button

Entry

Text

Menu

Listbox

Canvas

Frame

Checkbutton

Radiobutton

Events

Button Click

Keyboard

Mouse

bind()

command

Searching

Linear Search

Binary Search

Sorting

Bubble Sort

Quick Sort

Merge Sort

Queue

FIFO

Stack

LIFO

9. Most Important MCQs

1.

Tkinter is a _____.

- A) Database
- B) GUI Library
- C) Compiler
- D) Browser

Answer: B

2.

Which function creates the main window?

- A) main()
- B) Tk()
- C) start()
- D) open()

Answer: B

3.

Which function keeps the window running?

- A) execute()
- B) run()
- C) mainloop()
- D) show()

Answer: C

4.

Which widget accepts single-line input?

- A) Label
- B) Entry
- C) Text
- D) Canvas

Answer: B

5.

Which widget accepts multiple lines?

- A) Entry
- B) Button
- C) Text
- D) Label

Answer: C

6.

Which layout manager uses rows and columns?

- A) pack()
- B) grid()
- C) place()
- D) Frame

Answer: B

7.

Which layout manager uses x and y coordinates?

- A) pack()
- B) grid()
- C) place()
- D) Label

Answer: C

8.

Which widget draws graphics?

- A) Entry
- B) Canvas
- C) Frame
- D) Menu

Answer: B

9.

Which widget groups other widgets?

- A) Frame
- B) Button
- C) Label
- D) Entry

Answer: A

10.

Which widget allows only one option?

- A) Checkbutton

B) Radiobutton

C) Text

D) Menu

Answer: B
